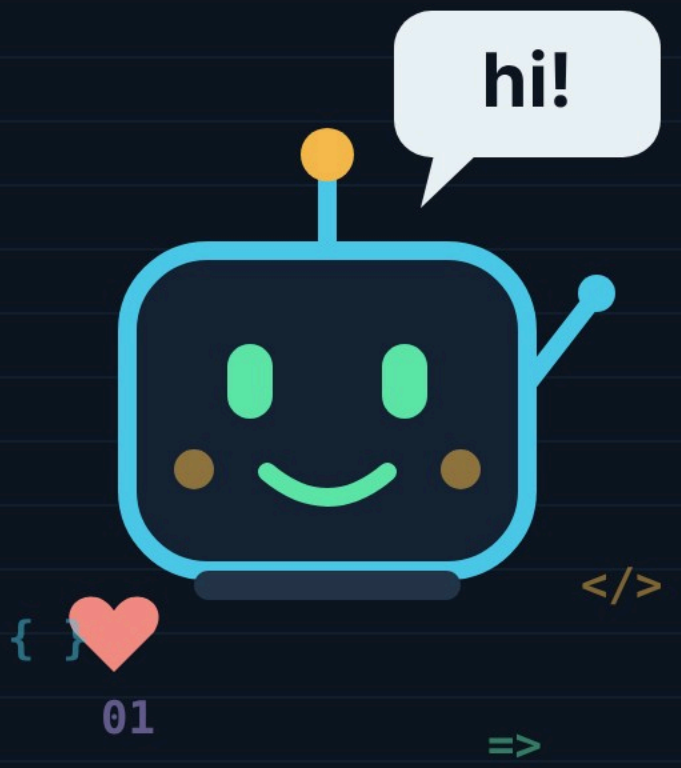
Hello, world!

```
> console.log("welcome to backend day")
welcome to backend day
```



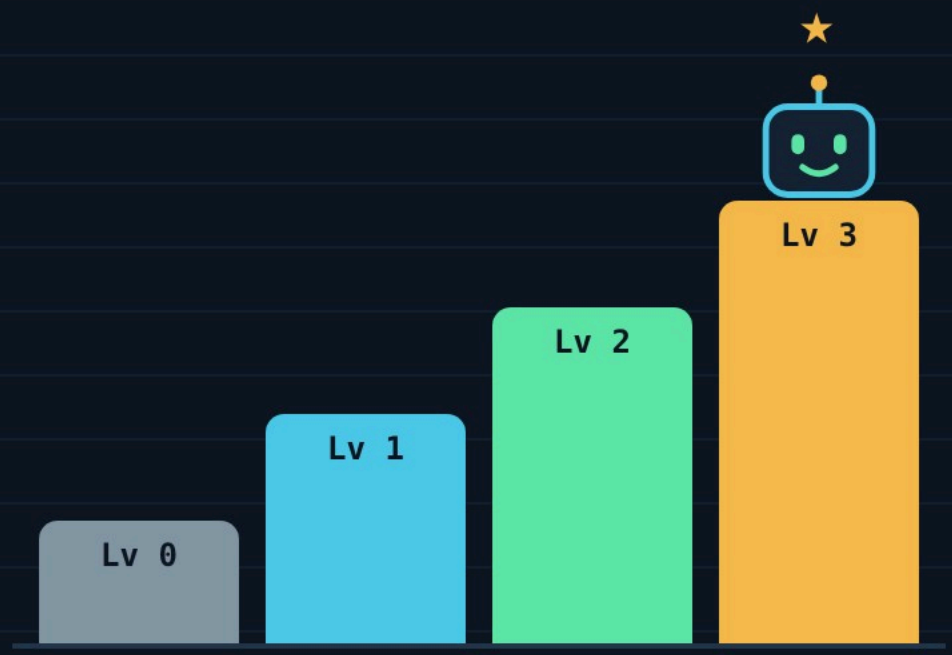
~/backend-day $ npm start

© Ashan Dimantha

POLL 00 · who's in the room

Have you built a back end before?

- **Lv 0** Not yet: "what does a server actually do?"
- **Lv 1** Followed a tutorial or two
- **Lv 2** Built an API for a project or assignment
- **Lv 3** Shipped one that real users depend on



Hands up for your level · every level is welcome today

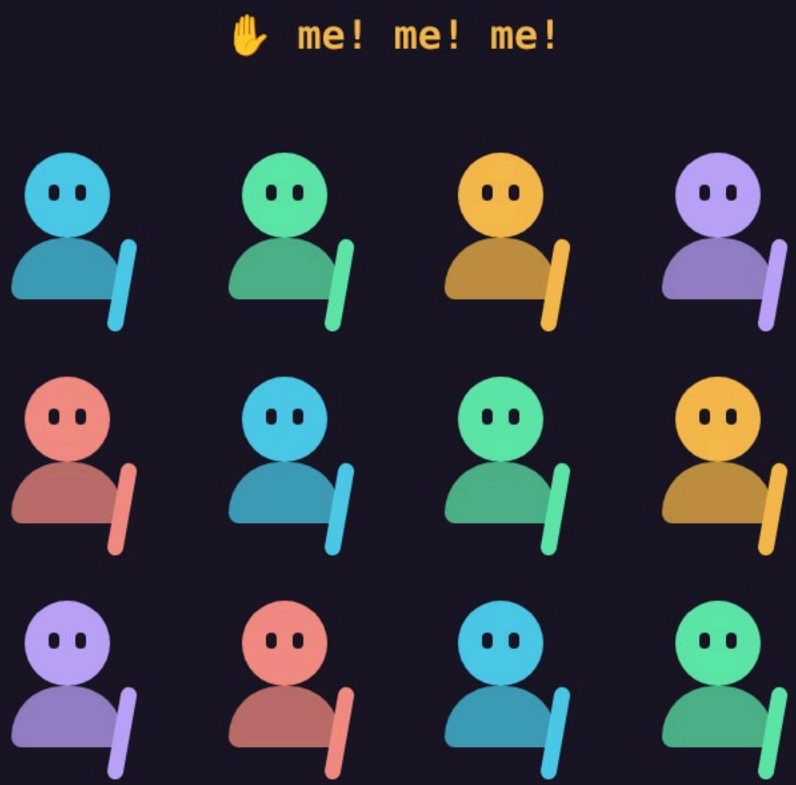
© Ashan Dimantha

HANDS UP 00 · quick show of hands

Who wants to be a full-stack engineer?

Keep your hand up while it's still true for you...

1. I've built a web page or an app screen
2. ...and the API behind it
3. ...and the database
4. ...and put it live on the internet



Today covers steps 2, 3 and 4

© Ashan Dimantha

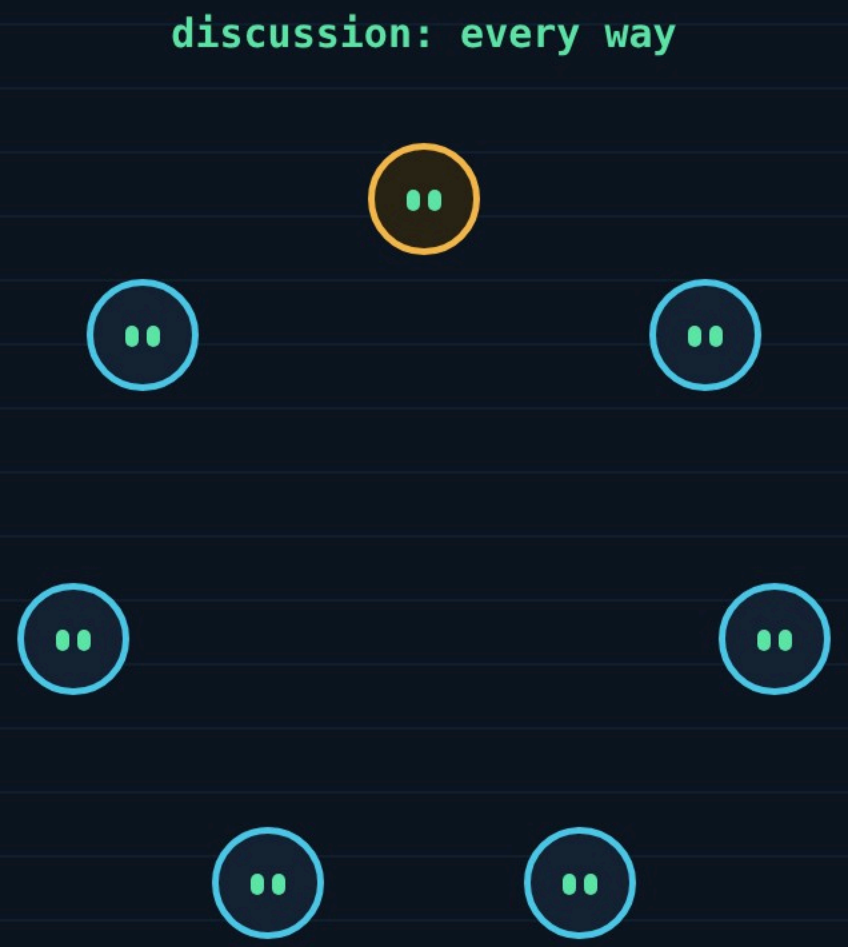
$ git checkout -b discussion

Not a lecture. A conversation.

Interrupt anytime. Questions are the plan.

Share how you do it at work. Disagree out loud.

No silly questions. Wrong answers teach the most.



You talk at least as much as I do today.

© Ashan Dimantha

$ cat my-promises.txt

What can you expect from me today?

01 · EXPLAIN

As much as I can

The why behind every concept, with everyday analogies. No question is too basic.

02 · HANDS-ON

You build it

Two labs: a real API, then a real database. You type the code yourself.

03 · ANSWER

Every question

Ask anytime. If I don’t know, we’ll find out together.

And from you: curiosity, questions, and a laptop that runs Node

© Ashan Dimantha

POLL 00 · language check

Which language should we talk in?

Hello! Ready to build an API?

English

raise 1 finger

ආයුබෝවන්! අපි API එකක් හදමු.

සිංහල

Sinhala · raise 2 fingers

Machan, API eka hadamu!

Singlish

raise 3 fingers

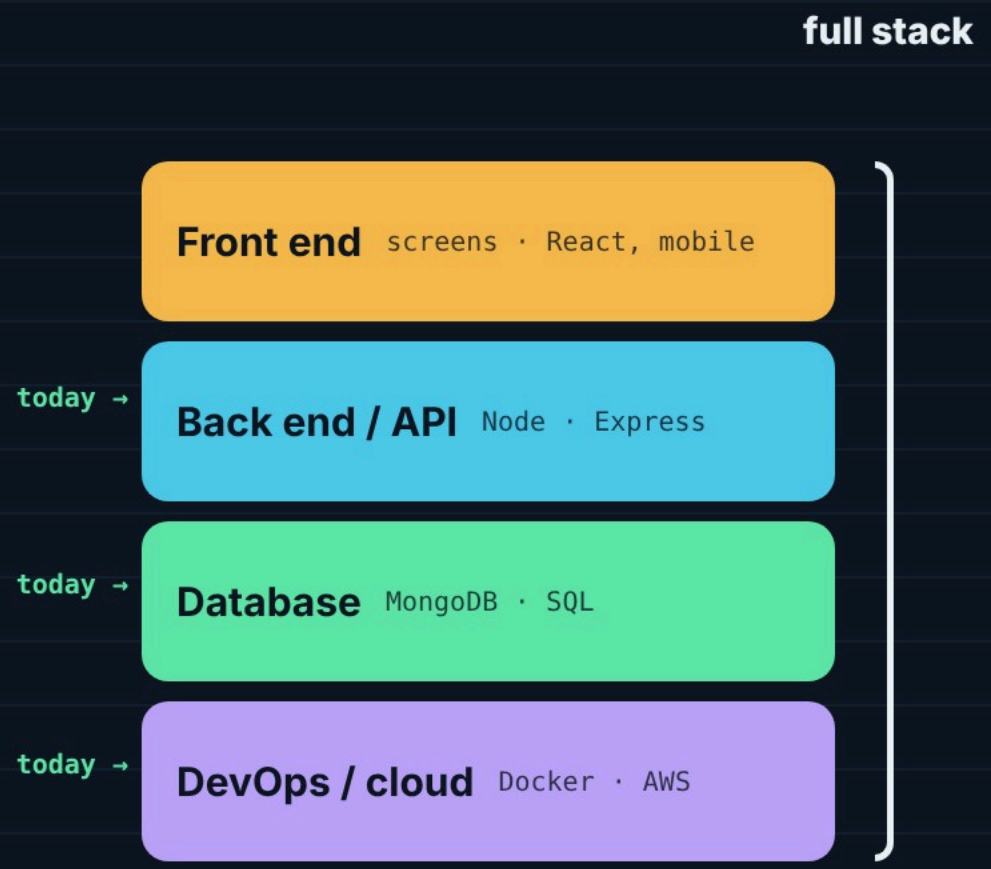
Mix freely · code and tech terms stay in English

© Ashan Dimantha

What is full-stack engineering?

Building a feature from the screen all the way down to the data, and getting it live.

- **Knows every layer** well enough to build and debug it
- **Deep in one**, comfortable in the rest (T-shaped)
- **Sees the whole flow**: a click becomes a request, a row, a response



Today we go deep on the back half of the stack

© Ashan Dimantha

Pick one from each layer: what's in a tech stack?

FRONT END

React · Angular · Vue
Next.js · Flutter

BACK END

Node + Express
Spring Boot · .NET
Django · FastAPI · Laravel

DATABASE

MongoDB
PostgreSQL · MySQL
Redis (cache)

SHIP IT

Docker · AWS
Azure · GCP
Vercel · GitHub Actions

FAMOUS COMBOS

MERN · MongoDB, Express, React, Node

MEAN · same, with Angular

Spring Boot + React/Angular + PostgreSQL

.NET + Angular + SQL Server

Highlighted = today's stack: the back half of MERN, plus Docker and AWS

© Ashan Dimantha

What shape of engineer are you?

Across = how broad your knowledge is · Down = how deep you go

I-shaped

Deep in one thing

The specialist: “the database person”

Dash / generalist

A little of everything

Great for prototypes, shallow in the hard parts

T-shaped

Deep in one, broad in many

Back-end expert who can still fix the UI

Pi-shaped

Deep in two

e.g. back end and cloud/DevOps

Comb-shaped

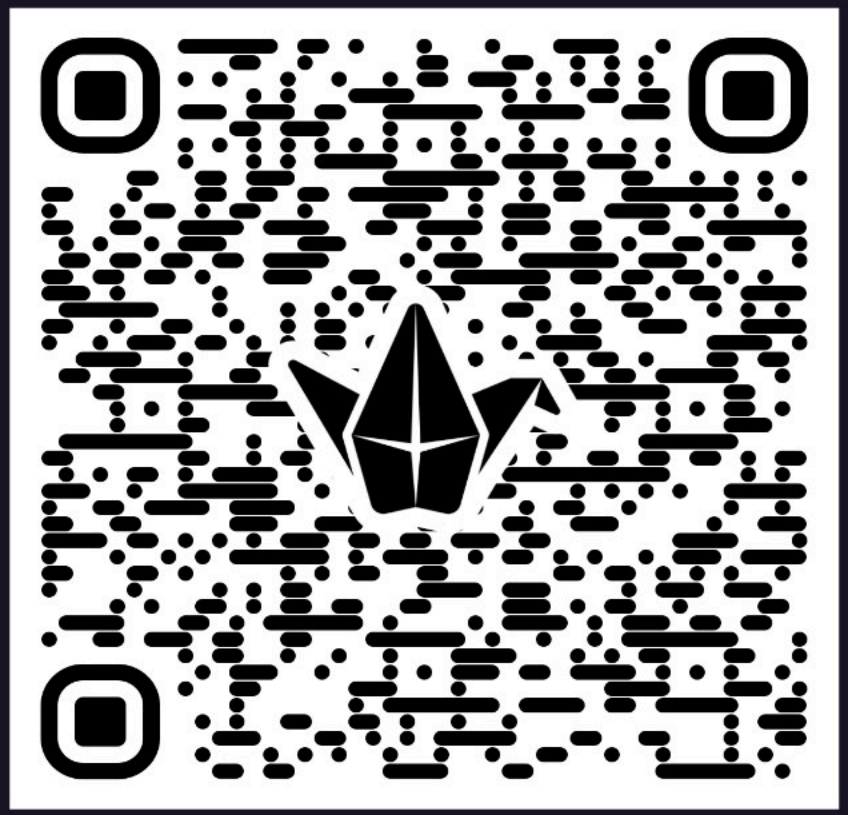
Deep in several

Grows over a career; rare early on

Today: go deep on back end, broad on the rest. That’s T-shaped.

© Ashan Dimantha

QnA and you answers



Vote: https://padlet.com/ashandimanthalk1/back-end-session-ousl-s0253w9fo47wojmp0uvf

© Ashan Dimantha

ACTIVITY 00 · Think-pair-share · 3 min

What’s the weirdest back-end bug you’ve ever met?

© Ashan Dimantha

dont’t worry

“everyone was a beginner one day”

“everyone faced huge struggles one day”

“everyone failed at least one day”

© Ashan Dimantha

...but

everyone didn't continue..
everyone didn't succeeded..

© Ashan Dimantha

MAP 00 · Welcome

How will we learn each idea?

01 WHY	02 WHAT	03 HOW	04 WHEN
The problem it solves	A plain definition	Step by step	Use it, or don't

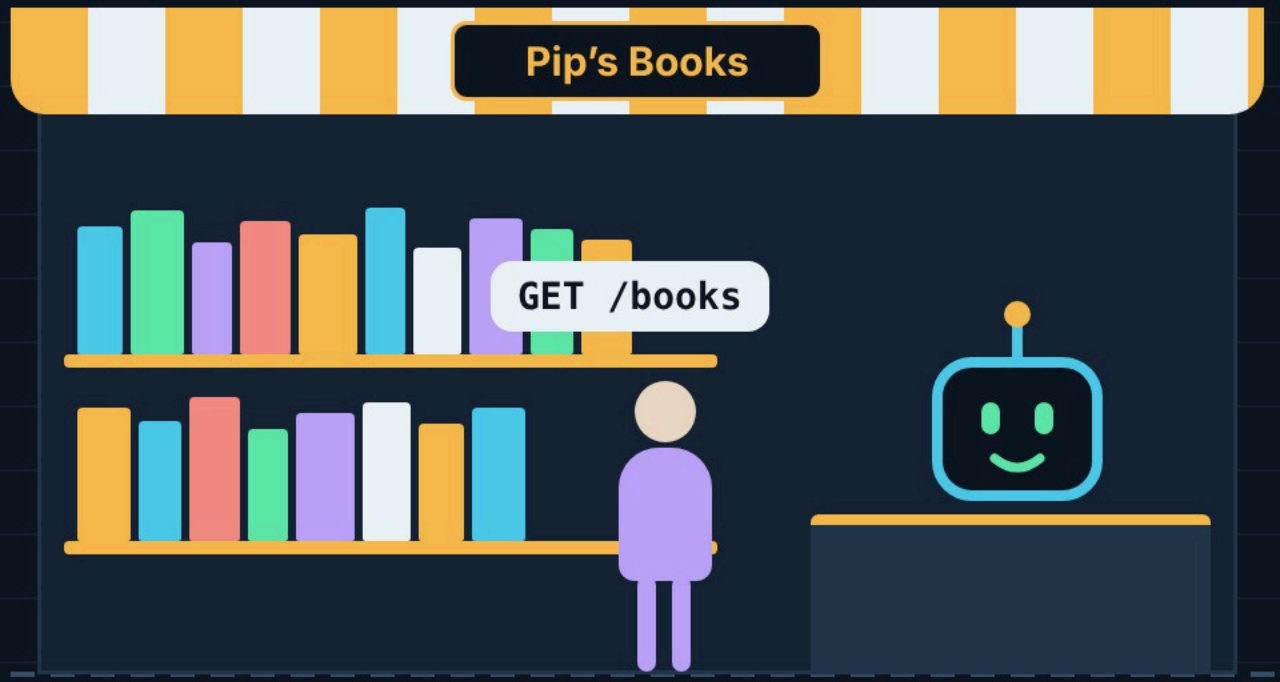
Every section opens with **a question for you**. You answer first, then the slide answers.

THEORY · listen & discuss LAB · laptops open ACTIVITY · your turn BREAK · stretch

© Ashan Dimantha

MAP 00 · Welcome · our project

Today we build the back end of a bookshop



behind the shop
the database
every book, saved

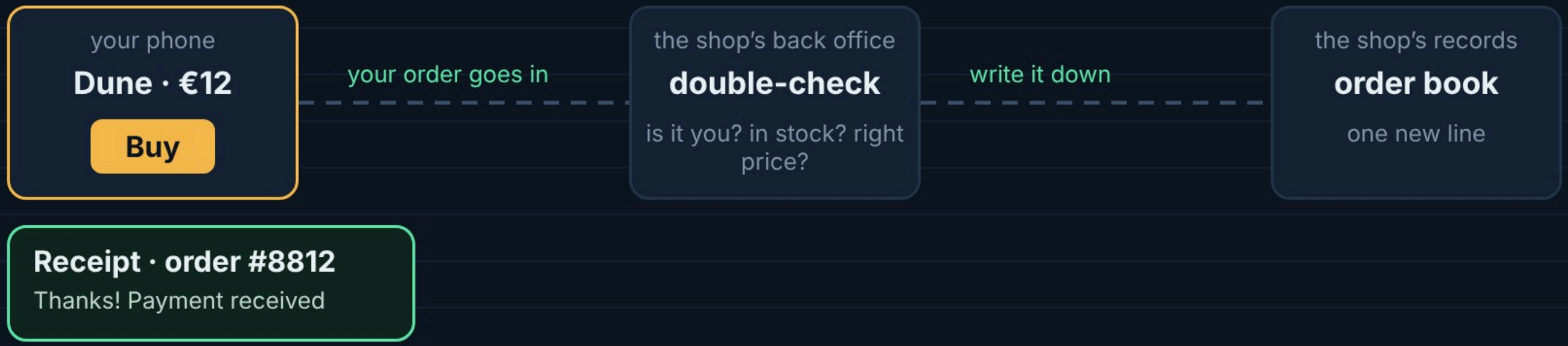
Customers = clients · Pip = our Express API · the shelf = MongoDB

© Ashan Dimantha

THEORY 01 · Intro to back-end · face a question

What happens between clicking “Buy” and a receipt appearing?

Shout out every step you can think of. Pip is counting.



~/backend-day/01-intro $

© Ashan Dimantha

I am

Ashan

Dimantha Herath

Software Engineer at Fortude
Technical Mentor at STEMLink
Content Creator

© Ashan Dimantha

$ ./backend-day --mode=hands-on --coffee=required

Back-End Engineering for Software Engineers

Hi, I'm Pip. I carry your requests.

Updates & Hands-On Foundations

Node.js | Express | MongoDB | Docker | AWS

09:00 → 16:00 · in person · [OUSL] · [07th Nov] · [Ashan Dimantha Herath]

© Ashan Dimantha

MAP 00 · Welcome

Where are we going today?

09:00 Welcome & icebreaker
09:10 Intro to back-end
09:30 Runtime & APIs
09:55 API design: SOAP · REST · GraphQL
10:25 Break
10:40 Networking basics
11:00 HTTP & status codes
11:20 Lab 1 · Build an Express API
12:05 Lunch
12:50 Controllers · services · repositories
13:15 Databases
13:35 Lab 2 · MongoDB CRUD
14:05 Break
14:20 Observability
14:35 Security & AI-era integration
14:55 Docker
15:15 Cloud deployment
15:35 Q&A · stump the presenter · Monday plan

Time colour = mode: cyan Theory · mint Lab · amber Activity · lilac Break

© Ashan Dimantha

BEFORE WE START 00 · Prerequisites

Are you ready for today?

KNOW

- JavaScript basics: variables, functions, if/else
- Arrays and objects (and arrow functions)
- Reading JSON
- Running commands in a terminal
- Nice to have: promises / async-await, basic git

INSTALL

- Node.js LTS (comes with npm)
- A code editor, e.g. VS Code
- An HTTP client: curl, Postman or similar
- Git
- Optional: Docker Desktop for the 14:55 demo

check in 30 seconds

```
$ node -v   $ npm -v   $ git --version
```

No database install needed: we hand you a link in Lab 2

~/backend-day $

VERIFY: current Node LTS

© Ashan Dimantha

THEORY 01 · Intro to back-end · face a question

What happens between clicking “Buy” and a receipt appearing?

Shout out every step you can think of. Pip is counting.

REVEAL · one possible answer

click → DNS lookup → TCP + TLS handshake → HTTP POST /orders → auth check → stock + payment rules → database write → payment provider call → 201 Created → receipt rendered

click	DNS	TCP+TLS	POST	auth	rules	DB write	payment	201	receipt

~/backend-day/01-intro $

© Ashan Dimantha

Why can’t the browser do it all?

Trust

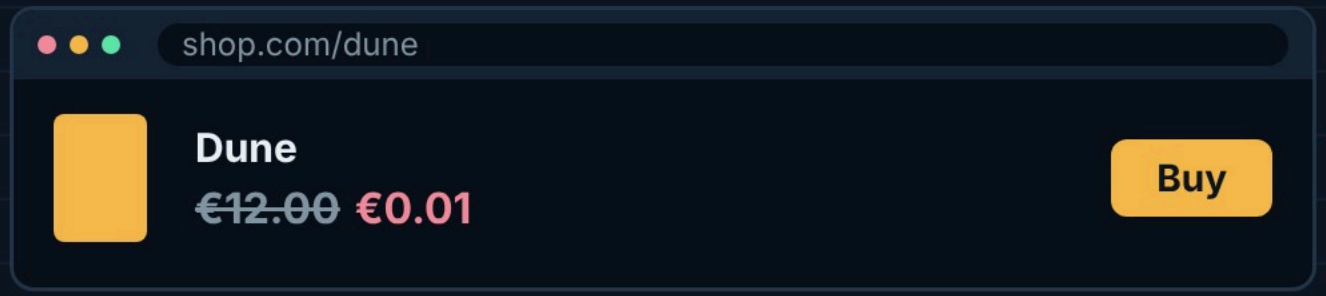
Users can change anything in their browser. The price must be checked somewhere they can’t touch.

Shared truth

Ten thousand shoppers must see the same last item in stock.

Secrets & heavy lifting

Payment keys, emails, reports and long jobs can’t live on a phone.

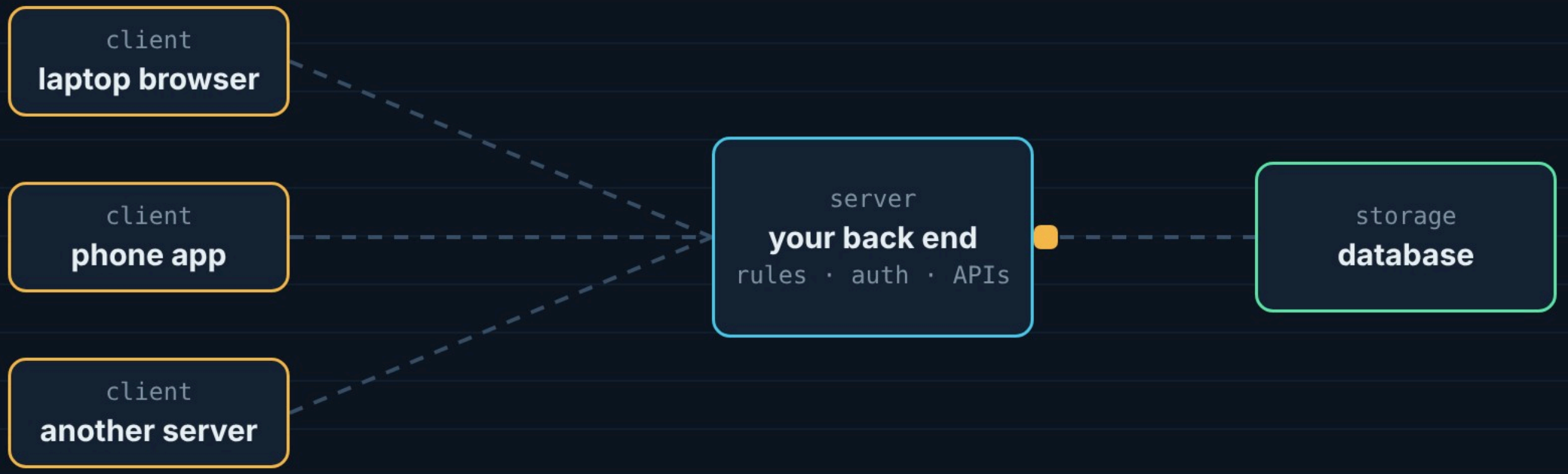


~/backend-day/01-intro $

© Ashan Dimantha

introduced somethig called

client-server architecture



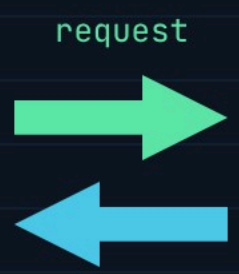
request · response · many clients ask, one server decides, the database remembers

© Ashan Dimantha

What's the difference between client and server?

Client

- Runs on the user's device
- Shows the UI, collects input
- Untrusted by default



Server

- Runs on machines you control
- Owns the data and the rules
- Serves many clients at once

RULE The **client starts** the conversation. The **server responds**.

© Ashan Dimantha

If a web app were a restaurant…

In the restaurant		In the back end
Diner	→	Client (browser, app)
Menu	→	API contract
Order slip	→	HTTP request
Kitchen	→	Server logic
Pantry	→	Database
Plate at the table	→	HTTP response

Chef Pip: “Diners never walk into my kitchen.”

© Ashan Dimantha

How does one order travel through the restaurant?

1 · The waiter takes your order to the kitchen

client sends an HTTP request

diner = client | menu = API contract | order slip = request | kitchen = server | pantry = database | plate = response

~/backend-day/01-intro $

© Ashan Dimantha

What does a back end actually do all day?

- Serves APIs
- Enforces business rules
- Stores data safely
- Knows who you are
- Talks to other services
- Runs background jobs

Plus the invisible jobs: logging, monitoring, scaling, staying up.

~/backend-day/01-intro $

© Ashan Dimantha

How does a request leave the browser?

START
Browser
User clicks “Buy”

→

STEP 1
DNS lookup
shop.com → 203.0.113.7

→

STEP 2
Connect
TCP + TLS handshake

→

STEP 3
Send
POST /orders + JSON

Post-office version: look up the address → reach the counter and show ID → hand over the sealed parcel.

© Ashan Dimantha

What happens inside the server?

STEP 4

Route

POST /orders → handler

STEP 5

Check rules

Auth, validation, stock

STEP 6

Persist

Write the order to the DB

STEP 7

Respond

201 Created + JSON

© Ashan Dimantha

Where should this logic live?

Concern	Client	Server
Input validation	Yes, for fast feedback	Always, for safety
API keys & secrets	Never	Yes
Price & discount maths	Display only	Calculate and decide
Who may do what	Hide the button	Enforce the rule
Date formatting	Yes	Optional

Rule of thumb: **the client is for experience, the server is for truth.**

© Ashan Dimantha

ACTIVITY 01 · Mini-quiz · 3 min

Quick check: true or false?

1 · A mobile app calling your API is a “client”.	True: client is a role
2 · The front end validates email, so the server can skip it.	False: never trust the client
3 · The TLS handshake happens before the DNS lookup.	False: DNS first, you need the IP

Thumbs up = true · thumbs down = false

© Ashan Dimantha

THEORY 02 · Runtime & APIs · face a question

JavaScript runs in your browser. Who runs it on a server?

There’s no browser on the server. So what’s doing the work?

REVEAL

A **runtime**: Node.js . It packages a JS engine, system APIs for files and network, and an event loop.

~/backend-day/02-runtime $

© Ashan Dimantha

What's inside the Node.js runtime?

your code · app.js	you write this
Node APIs · http fs crypto timers	talk to the world
V8 runs your JavaScript	libuv event loop + async I/O
operating system	sockets, files, threads

A runtime = an engine + the APIs + a scheduler that decides what runs next.

© Ashan Dimantha

When is Node.js a great fit, and when isn't it?

Great fit: I/O-bound

- JSON APIs over a database
- Real-time: chat, WebSockets
- Gateways that call other services

Careful: CPU-bound

- Image or video processing
- Heavy maths, big reports
- Fix: worker threads or a job queue

© Ashan Dimantha

Two teams build the app and the API at the same time. How do they not break each other?

REVEAL

They agree on an **API contract** first. Each side builds to the contract, not to each other's code.

© Ashan Dimantha

What does an API contract promise?

- Where: method + path
- What goes in: input shape
- What comes out: output shape
- What can go wrong: errors
- Who may call it: auth

```yaml
# openapi.yaml (excerpt)
paths:
  /books/{id}:
    get:
      summary: Get one book
      responses:
        '200': { description: The book }
        '404': { description: Not found }
```


© Ashan Dimantha

Why is a menu a contract?

On the menu		In the API
Dishes you can order	→	Endpoints
“No onions, extra cheese”	→	Parameters and body
The dish description	→	Response shape
“Sorry, sold out”	→	Error response
The new spring menu	→	A new API version

menu · the contract

Burger

beef patty, cheese

client app · expects the contract

reads patty = "beef"

Works as promised

Secretly changing what “Burger” means = a **breaking change**

© Ashan Dimantha

ACTIVITY 03 · API design · poll

09:55 → 10:25 · 30 min

Which API style does your team use most?

A	REST (Representational State Transfer) / JSON over HTTP	__%
B	GraphQL	__%
C	SOAP / XML	__%
D	gRPC, events, or “it’s complicated”	__%

60

Voting is open

60 seconds · scan the QR code or hold up A · B · C · D

Vote: [poll link or QR code] · or A/B/C/D fingers

© Ashan Dimantha

Why did each style appear?

~1998–2000

SOAP

Enterprises needed strict, tool-generated contracts between different languages and vendors.

2000

REST

Roy Fielding described the web's own style: simple, stateless, cacheable, built on HTTP.

2012 → 2015

GraphQL

Facebook's mobile apps needed exactly the right data in one round trip. Open-sourced 2015.

© Ashan Dimantha

What does a SOAP call look like?

- An XML envelope
- Contract in a WSDL(Web Services Description Language) file
- Standards for security and transactions
- Still alive in banks, telcos, government

```
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <GetBook xmlns="urn:example:books">
      <Id>42</Id>
    </GetBook>
  </soap:Body>
</soap:Envelope>
```

Analogy: a formal government form. Every box defined, nothing left to guesswork, lots of paperwork.

© Ashan Dimantha

How does REST (Representational State Transfer) name things?

- URLs are nouns: resources
- HTTP methods are the verbs
- Stateless: each request stands alone
- Cacheable: GETs can be cached

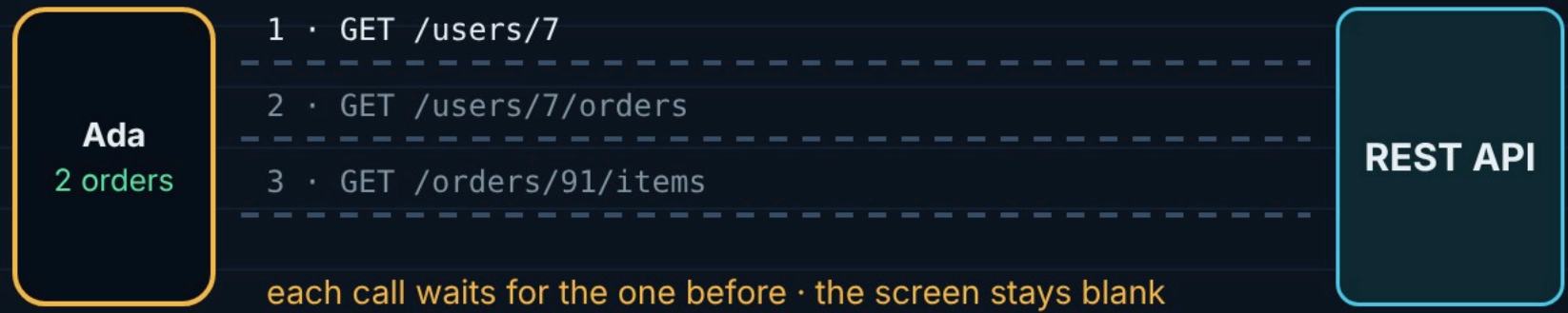
Method	Path	Means
GET	/books	list
GET	/books/42	read one
POST	/books	create
PATCH	/books/42	update part
DELETE	/books/42	remove

Analogy: a restaurant menu. Fixed dishes with clear names; you order by name.

© Ashan Dimantha

One profile screen, three REST calls. What hurts on a phone?

```
GET /users/7
GET /users/7/orders
GET /orders/91/items
```



Over-fetching

You get 40 fields. The screen shows 3.

Under-fetching

Three round trips on a slow network before anything renders.

© Ashan Dimantha

How does GraphQL ask for exactly enough?

```
# POST /graphql
query {
  user(id: "7") {
    name
    orders(last: 2) {
      total
    }
  }
}
```

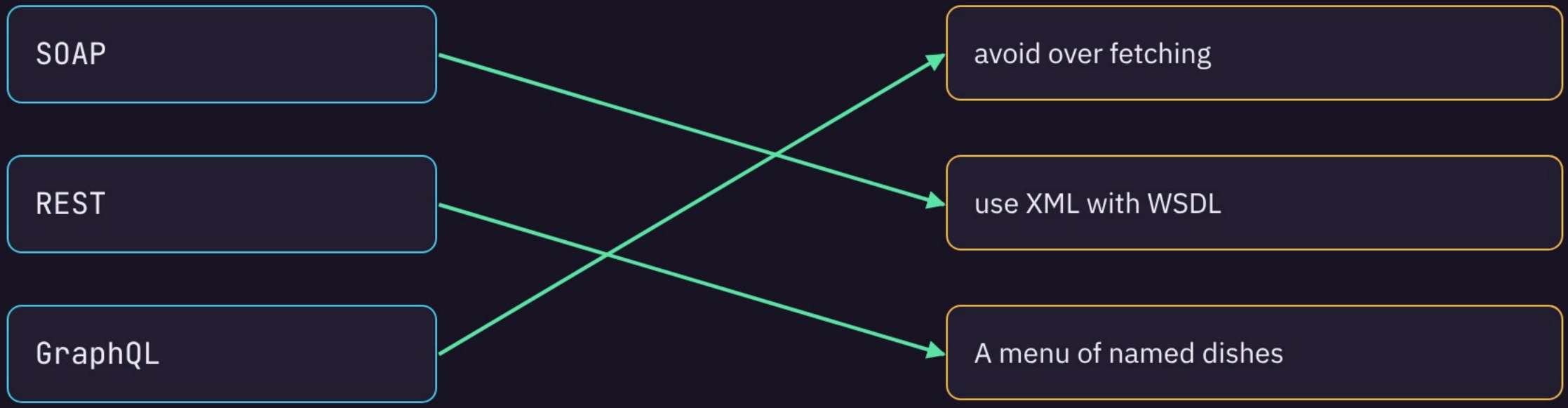
```
# 200 OK
{ "data": { "user": {
  "name": "Ada",
  "orders": [
    { "total": 42.5 },
    { "total": 18 }
] } } }
```

One endpoint · a typed schema · the client picks the fields.

© Ashan Dimantha

ACTIVITY 03 · Analogy match · 2 min

Match each style to its real-world twin



Call out a match, then click to draw the line

© Ashan Dimantha

How do they compare side by side?

	SOAP	REST	GraphQL
Payload	XML	Usually JSON	JSON
Contract	WSDL, required	OpenAPI, optional	Schema, required
Endpoints	One per service	One per resource	Usually one
HTTP caching	Hard	Easy, built in	Needs extra work
Best for	Enterprise integration	Public & CRUD APIs	Many clients, nested data
Watch out	Verbosity	Over/under-fetching	Query cost, N+1

© Ashan Dimantha

Which style should I pick?

1 · Must you talk to a system that already speaks SOAP? yes → SOAP

no ↓

2 · Many client types needing different shapes of nested data? yes → GraphQL

no ↓

3 · Everything else: public, CRUD, cache-friendly → REST (default)

Internal, high-throughput service-to-service? Also look at gRPC.

© Ashan Dimantha

ACTIVITY 03 · Think-pair-share · 4 min

You’re the architect. Which style, and why?

A · A payments API for thousands of outside developers	REST + OpenAPI
B · Web, mobile and watch apps sharing nested dashboards	GraphQL
C · Nightly sync with a bank’s long-running core system	SOAP, if that’s what it speaks

Think 1 min · pair 2 min · share 1 min · justify with one trade-off

© Ashan Dimantha

THEORY 04 · Networking · face a question

10:40 → 11:00 · 20 min

You type api.shop.com. How does your laptop find the right machine?

REVEAL · five pieces

DNS turns the name into an IP · **IP** routes packets to the machine · the **port** picks the program · **TCP** keeps it reliable · **TLS** keeps it private

Laptop	DNS	IP	Port	TCP	TLS
api.shop.com?	→ 203.0.113.7	hop, hop, hop	:443 → web app	handshake done	locked, private

~/backend-day/04-network $

© Ashan Dimantha

What do an IP address and a port point to?

203.0.113.7:443

IP ADDRESS = THE BUILDING

The street address of an apartment block. Finds the machine.

PORT = THE FLAT NUMBER

Which door inside. Finds the program listening there.

IPv4 looks like 203.0.113.7; IPv6 looks like 2001:db8::7. Same job, far more addresses.

© Ashan Dimantha

How does DNS find the address?

1. Check the local cache
2. Ask a resolver (ISP, public)
3. Resolver walks root → .com → shop.com
4. Answer cached for its TTL

Analogy: your phone's contacts. Check your own list first; if it's not there, call directory enquiries, who look it up and you save it for later.

```
$ dig +short example.com try it: nslookup works too
```


© Ashan Dimantha

ACTIVITY 04 · Which port? · 2 min

Which door does each service usually use?

HTTP	80
HTTPS	443
SSH	22
PostgreSQL	5432
MongoDB	27017
Node/Express	3000/3080

These are defaults and conventions. Every one can be changed.

© Ashan Dimantha

THEORY 05 · HTTP · face a question

11:00 → 11:20 · 20 min

If you could open an HTTP request like a letter, what would you find inside?

```
POST /books HTTP/1.1
Host: api.shop.com
Content-Type: application/json
· blank line ·
{"title": "Dune"}
```

← address line

← envelope notes

← then a gap

← the letter

REVEAL

An address line (method + path), envelope notes (headers), a blank line, and the letter itself (body). In HTTP/1.1 it's literally text.

~/backend-day/05-http $

© Ashan Dimantha

What does a request and its response look like?

```
# request
POST /books HTTP/1.1
Host: api.shop.com
Content-Type: application/json
Authorization: Bearer eyJhbGci…

{"title": "Dune"}
```

```
# response
HTTP/1.1 201 Created
Location: /books/42
Content-Type: application/json

{"id": 42, "title": "Dune"}
```

start line · headers · blank line · body

© Ashan Dimantha

Same address, different wish: what is an HTTP method?

the first line of every request

```
GET /books/42 HTTP/1.1
  ↑ the verb    ↑ the noun
```

The URL says **which thing**. The method says **what you want done** to it.

At the library desk, about book 42, you can ask to...

GET	POST	PUT	PATCH	DELETE
look at it	donate a new book	swap in a whole new copy	fix one torn page	take it off the shelf

~/backend-day/05-http $

© Ashan Dimantha

Which verb should I use?

Method	Does	Safe?	Idempotent?
GET	Read	Yes	Yes
POST	Create, or run an action	No	No
PUT	Replace the whole thing	No	Yes
PATCH	Change some fields	No	Not guaranteed
DELETE	Remove	No	Yes

Safe = doesn't change anything. **Idempotent** = doing it twice ends the same as once.

© Ashan Dimantha

What goes in the body, and what can go wrong?

Method	Body?	Example	Security: watch out for
GET	No: data rides in the URL	/books?title=Dune	URLs get logged and saved: never put secrets in them
POST	Yes: the new thing	/books + {title}	Validate every field; a double click can create two
PUT	Yes: the whole thing	/books/2 + full book	Missing fields get wiped; check who owns it
PATCH	Yes: only what changes	/books/2 + {year}	Allow-list fields, or someone sends "role":"admin"
DELETE	Usually none: id in path	/books/2	Check permission first; consider a soft delete

HTTPS hides the URL and the body on the wire, but URLs still land in logs and history

Every method except GET changes data: each one needs auth and validation

© Ashan Dimantha

How does the server tell us how it went?

the first line of every response

```
HTTP/1.1 404 Not Found
         ↑ for code  ↑ for humans
```

A **status code** is a three-digit answer to “how did it go?”. Programs read the number; people read the words.

THE FIRST DIGIT

tells you the family: 2 = good, 4 = your fault, 5 = our fault

THE OTHER TWO

say exactly what: 404 not found, 401 who are you?

WHY CARE

Apps, retries, alerts and caches all decide by this number

⚠ Never answer **200** with {"error": "..."} inside: every tool will think it worked.

© Ashan Dimantha

What is the airport telling you?

Range	Official name	Means	At the airport
100–199	Informational	Hold on, still going	“Boarding soon, please wait”
200–299	Successful	It worked	“You’re on board. Enjoy the flight”
300–399	Redirection	Look elsewhere	“Gate changed, go to B12”
400–499	Client Error	Your mistake	“Wrong ticket, or no passport”
500–599	Server Error	Our mistake	“The airline’s system is down”

The first digit is the answer to: **whose fault, and what next?**

© Ashan Dimantha

Which 15 codes cover most APIs?

2xx · 3xx

200 OK

201 Created

204 No Content

301 / 308 Moved

304 Not Modified

4xx · client

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Unprocessable

429 Too Many Requests

5xx · server

500 Internal Error

502 Bad Gateway

503 Unavailable

© Ashan Dimantha

$ theory --done && hands --on-keyboard

Let's really start.

> laptops open · terminals ready · code time

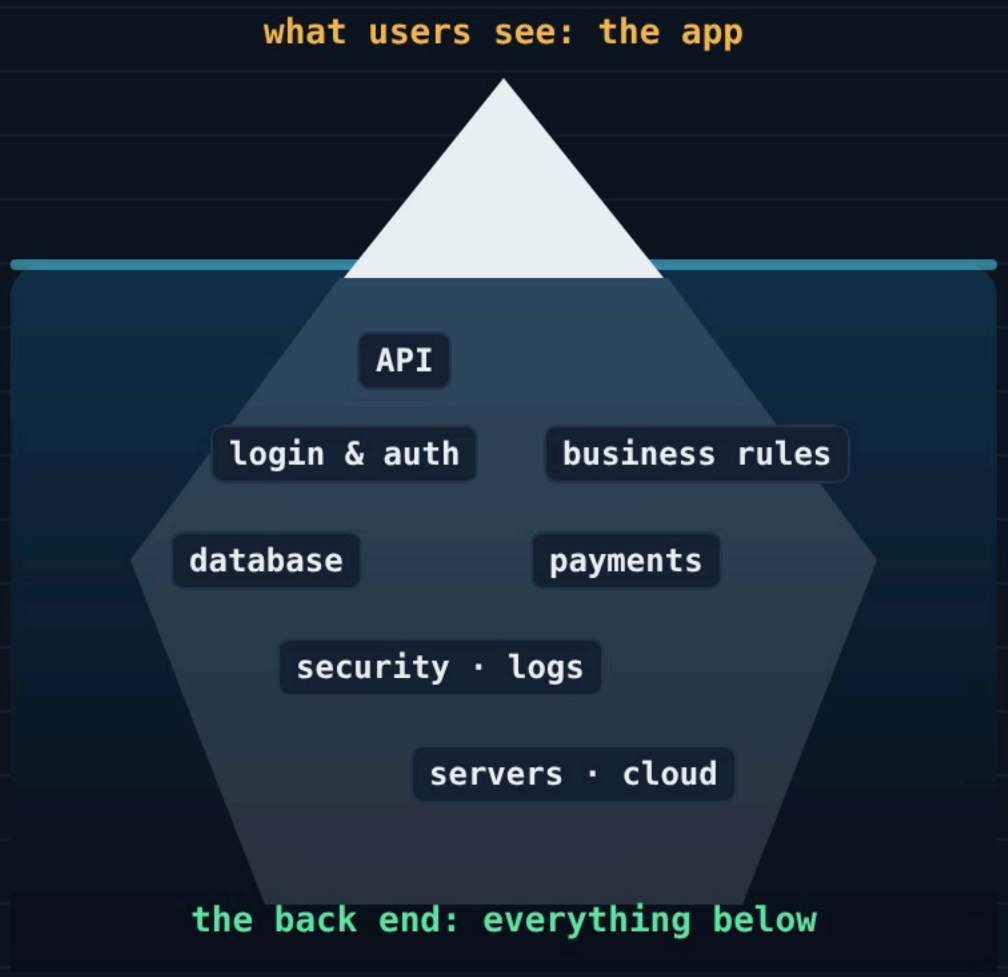
~/backend-day $ mkdir books-api

© Ashan Dimantha

What is back-end engineering?

Building the part of an app users never see, but always depend on.

- **Receive** requests through an API
- **Decide**: business rules, logins, permissions
- **Remember**: store and find data safely
- **Keep it running**: fast, secure, always on



Front end = what you see · Back end = what makes it work

© Ashan Dimantha

What can you build a back end with?

Language	Popular frameworks	Known for
JavaScript / TS	Express · NestJS · Fastify	One language front to back; fast to start (today)
Java	Spring Boot	Big companies, banks, long-lived systems
C#	ASP.NET Core	Enterprise and Microsoft/Azure shops
Python	Django · FastAPI	Quick to write; the home of data and AI
Go	standard library · Gin	Small, fast services and cloud tools
PHP	Laravel	Web apps and much of the existing web

Plus the same pieces in every stack: a **database**, a **cache**, sometimes a **queue**, and a way to **deploy**.

Different syntax, same ideas: routes, layers, status codes, databases

© Ashan Dimantha

LAB 06 · Lab 1 · Build an API with Node & Express

11:20 → 12:05 · 45 min

Lab 1: can we build a Books API in 45 minutes?

YOU'LL BUILD

- GET /books and /books/:id
- POST /books with validation
- A 404 and a 500, on purpose

YOU NEED

- Node.js LTS + npm
- A terminal and an editor
- curl or any HTTP client

HOW WE WORK

- Pairs: driver + navigator
- Swap at step 4
- Checkpoint after every step

7 steps · 1 spot-the-bug · solution hidden until 11:58

```
$ curl localhost:3000/books
[{"id":1,"title":"Dune"}]
```

LAB 1 PROGRESS

1 2 3 4 5 6 7

Books API is live ✓

~/backend-day/lab1 $

VERIFY: Node LTS + Express major

© Ashan Dimantha

What is Node.js, in one slide?

WHAT

JavaScript, running outside the browser

INSIDE

V8 engine + system APIs + an event loop

GOOD AT

Lots of waiting: APIs, databases, files

COMES WITH

npm: a huge library of packages

It even has a built-in **http** module. So why would we need anything else? Let’s find out.

© Ashan Dimantha

What is npm, and why do we need it?

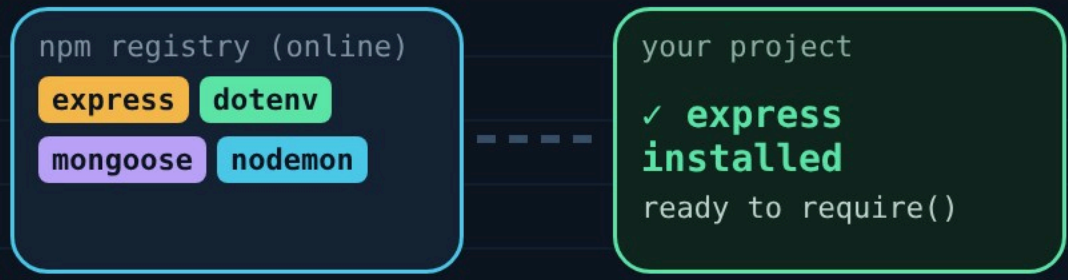
WHAT

npm = Node Package Manager

A huge online library of ready-made code (**packages**), plus one command to download them.

It comes free with Node. Installed Node? You have npm.

```
$ npm install express
```



WHY

Don’t reinvent the wheel

Need a web server? Someone already built Express

One command

No hunting for zip files or copying code by hand

Same for the whole team

A teammate runs npm install and gets the same packages

npm = an app store for code · free, and it came with Node

© Ashan Dimantha

ACTIVITY 06 · Lab 1 · before we type

Can we switch the AI helpers off for the labs?

- Inline autocomplete — OFF
- AI chat in the editor — OFF
- Coding agent in the terminal — OFF
- Chatbot tab in the browser — OFF

Brain: on. You're the engine today.

WHY
You learn by typing it and breaking it yourself

STUCK?
Ask your pair, the slides, then us

AFTER 16:00
Turn them back on and ask them to review your code

Thumbs up when yours are off

© Ashan Dimantha

Is Node ready on your laptop?

1 · CHECK

```
$ node -v
$ npm -v
```

Two version numbers? Skip to 3.

2 · INSTALL (IF NEEDED)

Easiest: the **LTS** installer from nodejs.org

Or a version manager:

```
$ nvm install --lts
```

3 · PROVE IT

```
$ node -e "console.log('hi')"
hi
```

Node runs JavaScript. You're set.

WHY LTS?

Long-Term Support: stable and patched for years

ALSO HAVE

An editor, a terminal, and curl or an HTTP client

~/backend-day/lab1 $

VERIFY: current Node LTS

© Ashan Dimantha

How do we create a Node project?

```
# terminal · one line at a time
$ mkdir books-api # new folder
$ cd books-api    # go inside
$ npm init -y
Wrote to …/books-api/package.json
$ touch server.js # or New File
$ code .          # open editor
```

```
YOUR PROJECT
books-api/
├─ package.json ← npm
└─ server.js    ← you
```

```
# package.json
{ "name": "books-api",
  "version": "1.0.0",
  "scripts": { … } }
```

WHAT IS package.json?

Your project's ID card: name, version, scripts, and the packages it needs

WINDOWS?

No touch: use New-Item server.js, or New File in VS Code

Checkpoint: your editor shows package.json and an empty server.js.

~/backend-day/lab1 $

© Ashan Dimantha

How do we give our app settings without changing its code?

Like an oven: the recipe stays the same, you just set the dial. **Environment variables** are the dials.

```
# in the code: read the dial
const port = process.env.PORT || 3000;
server.listen(port);
```

```
# in the terminal: set the dial
$ PORT=4000 node server.js
# Windows PowerShell
$env:PORT=4000; node server.js
```

process.env holds every variable, always as text

|| 3000 is the default when nobody sets it

In Lab 2 we keep them in a **.env** file

~/backend-day/lab1 $

© Ashan Dimantha

require or import: what's the difference?

COMMONJS · Node's original (today)

```
const express = require('express');
module.exports = router;
```

ES MODULES · the JavaScript standard

```
import express from 'express';
export default router;
```

	require	import
Loads	while the code runs, anywhere	before the code runs, at the top
Default in Node	yes, for .js files	with "type": "module" or .mjs
Works in browsers	no	yes
Mix them?	one style per file	one style per file

Same job: bring code from another file or package. Pick one style per project.

© Ashan Dimantha

How do we switch our project to import?

1 · TELL NODE

```
# package.json
"type": "module",
```

Every .js file is now an ES module

2 · SWAP THE LINES

```
- const express = require('express');
+ import express from 'express';
- module.exports = router;
+ export default router;
```

3 · ADD .js TO YOUR FILES

```
import books from
  './routes/books.js';
import 'dotenv/config';
```

Packages: no extension needed

```
# app.js, ES module version
import 'dotenv/config';
import express from 'express';
import books from './routes/books.js';
const app = express();
app.use(express.json());
app.use('/books', books);
```

Change every file at once: one style per project

© Ashan Dimantha

Can we build a server with only Node?

```
# server.js · no packages, just Node
const http = require('node:http');
const server = http.createServer((req, res) ⇒ {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ status: 'ok' }));
  }
  res.writeHead(404).end();
}); server.listen(3000);
```

It works: node server.js, then curl localhost:3000/health. Yes, really.

~/backend-day/lab1 $

© Ashan Dimantha

Why Express?

Node is an empty kitchen with gas and water. Express adds the stations, the order rail and a system everyone already knows.

ROUTING
app.get('/books/:id') with params done for you

MIDDLEWARE
Plug-in steps: JSON parsing, auth, logging

HELPERS
res.status(201).json(book) in one line

ONE ERROR HANDLER
Bad input becomes a 400, not a crash

Small, unopinionated, the most widely used Node web framework. Huge ecosystem and docs.

~/backend-day/lab1 $

© Ashan Dimantha

So what is Express, really?

Express is a small library that sits on top of Node's own http module. It doesn't replace Node; it gives the raw request and response a friendlier shape, so we stop parsing URLs and writing headers by hand.

Every request travels down a line of small functions called **middleware**. Each one can read the request, add something to it, answer it, or pass it on with next(). Parsing JSON, logging and checking a login are all just middleware.

At the end of the line is the **route handler**: the one function for this method and path, like GET /books/:id. If anything throws on the way, Express jumps straight to the error handler.

© Ashan Dimantha

How do we get Express onto our laptop? Meet npm

> npm is a supermarket for code: millions of free packages, and a tool to fetch them.

package.json

The shopping list: name, scripts, and which packages you need

package-lock.json

The receipt: the exact versions you got, so everyone gets the same

node_modules/

The pantry: the downloaded code. Never commit it.

```
$ npm init -y          # start a shopping list
$ npm install express # add to the list and fetch it
$ npm install         # fetch everything on the list
```

~/backend-day/lab1 $

© Ashan Dimantha

The same two endpoints, now in Express

```
# app.js · npm install express
const express = require('express');
const app = express();
app.use(express.json());
app.get('/health', (req, res) ⇒ res.json({ status: 'ok' }));
app.post('/books', (req, res) ⇒ res.status(201).json({ id: 3, title: req.body.title }));
app.listen(3000);
```

PLAIN NODE
~20 lines · a crash on bad JSON

EXPRESS
6 lines · bad JSON gets a 400

Express still runs on Node’s http module underneath. It just does the boring parts for you.

© Ashan Dimantha

LAB 06 · Lab 1 · our two packages

What are we about to install, and why?

```
$ npm install express dotenv
```

express

The web framework: routes, middleware, JSON helpers

Our kitchen stations and order rail

```
const express = require('express');
```

dotenv

Reads a .env file into process.env

The settings dials, kept in one file

```
require('dotenv').config();
```

One command, two packages, both added to package.json. Next slide: we type it.

~/backend-day/lab1 $

© Ashan Dimantha

Step 1 · Can we scaffold a project?

```
$ mkdir books-api && cd books-api
$ npm init -y
$ npm install express dotenv
added packages, found 0 vulnerabilities
```

Checkpoint: package.json lists express and dotenv under dependencies, and a node_modules folder exists.

~/backend-day/lab1 $

© Ashan Dimantha

Step 2 · Is the server alive?

```
# app.js
const express = require('express');
const app = express();
app.get('/health', (req, res) ⇒ res.json({ status: 'ok' }));
app.listen(3000, () ⇒ console.log('API on http://localhost:3000'));
```

```
$ node app.js
$ curl localhost:3000/health
```

Checkpoint: you see {"status":"ok"}

~/backend-day/lab1 $

© Ashan Dimantha

Step 2b · Can we start the app with one short command?

```
# package.json · update "scripts"
"scripts": {
  "start": "node index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
}
```

then run

```
$ npm start
```

same as typing

```
$ node index.js
```

SCRIPTS
Named shortcuts for commands your team runs often

"start"
The standard name: hosts and Docker look for it

npm start
No "run" needed for start and test; others use npm run name

Checkpoint: Ctrl+C the server, run npm start, and /health still answers.

~/backend-day/lab1 $

© Ashan Dimantha

Step 3 · Can we list books?

```
# app.js · new lines highlighted
const express = require('express');
const app = express();
const books = [{ id: 1, title: 'Dune' }, { id: 2, title: 'Neuromancer' }];
app.get('/health', (req, res) ⇒ res.json({ status: 'ok' }));
app.get('/books', (req, res) ⇒ res.json(books));
app.listen(3000, () ⇒ console.log('API on http://localhost:3000'));
```

Checkpoint: curl localhost:3000/books returns both books as a JSON array.

~/backend-day/lab1 $

© Ashan Dimantha

Step 1b · Can the server restart itself every time we save?

```
# 1 · install it, for development only
$ npm install -D nodemon
```

```
# 2 · package.json, add a script
"scripts": {
  "dev": "nodemon app.js"
}
```

```
# 3 · from now on, start with
$ npm run dev
```

app.js

```
$ npm run dev
[nodemon] watching *.js
API on http://localhost:3000
… you edit app.js and press save …
[nodemon] restarting due to changes…
API on http://localhost:3000
no Ctrl+C, no retyping
```

What: watches your files and restarts the server on save

-D: a dev tool, not needed in production

~/backend-day/lab1 $

© Ashan Dimantha

Step 3b · Can we find a single book?

```
# app.js · add below GET /books
app.get('/books', (req, res) ⇒ res.json(books));
app.get('/books/:id', (req, res) ⇒ {
  const book = books.find((b) ⇒ b.id === Number(req.params.id));
  res.json(book);
});
```

:id
A blank in the path. /books/2 fills it with "2"

req.params.id
Where Express puts it, always as text

books.find
Returns the first match, or nothing

Checkpoint: curl localhost:3000/books/2 returns just Neuromancer. Now try /books/99...

~/backend-day/lab1 $

© Ashan Dimantha

Path params or query params: where does the extra info go?

```
/books/2          /books?title=Dune&sort=year
      ↑ path param              ↑ query params, after the ?
```

PATH PARAM · which one?

Names one thing. Part of the address, so it's required.

`route: /books/:id`

`read: req.params.id → "2"`

QUERY PARAMS · which ones, how?

Filter, sort or page a list. Optional, any order.

`route: /books (no change)`

`read: req.query.title → "Dune"`

Rule of thumb: removing it changes **which** resource → path. Only changes **the view** → query.

~/backend-day/lab1 $ · bonus challenge in the notes

© Ashan Dimantha

Step 4 · What if the book doesn't exist?

```
# app.js · add one line to GET /books/:id
app.get('/books/:id', (req, res) ⇒ {
  const book = books.find((b) ⇒ b.id ═══ Number(req.params.id));
  if (!book) return res.status(404).json({ error: 'Book not found' });
  res.json(book);
});
```

Checkpoint: /books/1 returns Dune. curl -i /books/99 shows 404 Not Found.

© Ashan Dimantha

What is Postman, and why not just use the browser?

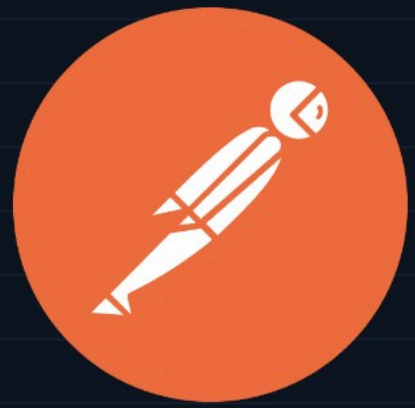
A browser's address bar only sends GET. Postman is a **remote control for APIs**: any method, any header, any body.

1 GET ▾ http://localhost:3000/books Send

2 Params Headers Body Auth

3 Response 200 OK 8 ms · 74 B

```
[{ "id": 1, "title": "Dune" }, { "id": 2, … }]
```



Request · one saved call

Collection · a folder of calls to share

Environment · swap localhost for a real server

~/backend-day/lab1 $ · first try: GET /books → 200

VERIFY: Postman local access

© Ashan Dimantha

Step 5 · Can clients add a book?

```
app.use(express.json()); // right after const app = express();
app.post('/books', (req, res) ⇒ {
  const { title } = req.body ?? {};
  if (!title) return res.status(400).json({ error: 'title is required' });
  const book = { id: books.length + 1, title };
  books.push(book);
  res.status(201).json({status: "ok"});
});
```

Checkpoint: the server starts without errors. We test it in step 6.

~/backend-day/lab1 $

© Ashan Dimantha

Step 6 · Does it answer like we promised?

```
$ curl -i localhost:3000/books/99
HTTP/1.1 404 Not Found
$ curl -i -X POST localhost:3000/books -H "Content-Type: application/json" -d '{"title":"Snow
Crash"}'
HTTP/1.1 201 Created
$ curl -i -X POST localhost:3000/books -H "Content-Type: application/json" -d '{}'
HTTP/1.1 400 Bad Request
```

Checkpoint: you saw 404, 201 and 400. GET /books now lists Snow Crash.

~/backend-day/lab1 $

© Ashan Dimantha

LAB 06 · Lab 1 · help desk

Stuck? Find your error here

You see	Try this
`EADDRINUSE :3000`	Another server is running. Ctrl+C it, or use port 3001.
`Cannot find module 'express'`	Run npm install inside the project folder.
title is always missing	express.json() goes before routes; send Content-Type: application/json.
Changes don’t show	Restart, or run node --watch app.js.
Every route returns 404	Check the path spelling; the 404 handler must be last.
curl acts strange on Windows	Use curl.exe, Git Bash, or an HTTP client app.

Still stuck? Red sticky note on your laptop. Help is coming.

© Ashan Dimantha

ACTIVITY 06 · Lab 1 · coding challenge · 5 min

5:00

Can you make POST /books answer with all the books?

```
# app.js · your POST route from step 5
app.post('/books', (req, res) ⇒ {
  const { title } = req.body ?? {};
  if (!title) return res.status(400).json(…);
  const book = { id: books.length + 1, title };
  books.push(book);
  res.status(201).json(book);  // ← change me
});
```

TODAY IT RETURNS

```
{ "id": 3, "title": "…" }
```

MAKE IT RETURN

```
[ Dune, Neuromancer, your book ]
```

still with status 201

Hint: which variable already holds every book?

Test: POST your favourite book in Postman, then look at the body

Answer on the next slide · green sticky note when it works

© Ashan Dimantha

Answer · one word changes

```
# app.js
app.post('/books', (req, res) => {
  const { title } = req.body ?? {};
  if (!title) return res.status(400).json(…);
  const book = { id: books.length + 1, title };
  books.push(book);
  res.status(201).json(books);
});
```

```
response · 201 Created
[
  { "id": 1, "title": "Dune" },
  { "id": 2, "title": "Neuromancer"
},
  { "id": 3, "title": "Your book" }
]
```

WHY IT WORKS

push() runs first, so books already holds the new one

BUT IN REAL APIS

Usually return just the new book: the list can grow to millions

© Ashan Dimantha

Your 300-line route handler works. Why would anyone split it up?

REVEAL

Because tomorrow someone must change it, test it, or swap the database, without breaking everything else.

© Ashan Dimantha

Why structure a project at all? It works in one file…

One cook can run a food cart. A restaurant needs stations, or dinner never leaves the kitchen.

1 · FIND IT

Every piece of code has one obvious home

2 · CHANGE IT SAFELY

A new rule touches one file, not twenty

3 · TEST IT

Check the rules without a server or a database

4 · SHARE IT

Teammates work on different parts without clashing

5 · SWAP IT

Change the database without rewriting the routes

6 · GROW IT

5 routes or 500: same shape, faster onboarding

But: a 50-line script doesn't need layers. Structure pays off as the project and the team grow.

© Ashan Dimantha

Who are controller, service and repository?

Controller

Talks HTTP

Reads the request, calls the service, sends the response and status code.

`controllers/bookController.js`

Service

Decides

Holds the business rules: what's allowed, what to calculate, what happens next.

`services/bookService.js`

Repository

Remembers

Saves and finds data. The only part that knows which database we use.

`repositories/bookRepository.js`

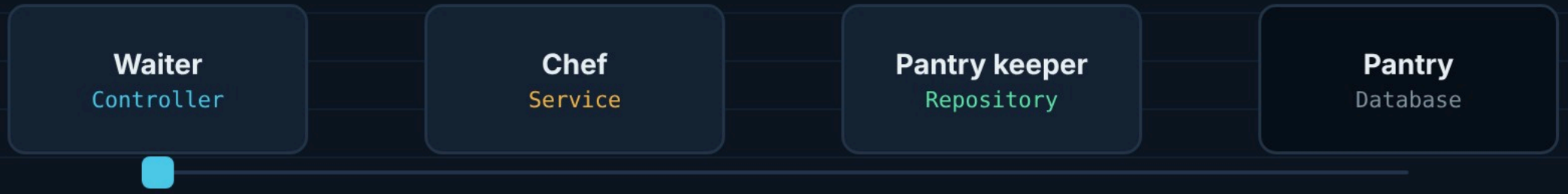
Controller talks · Service decides · Repository remembers

~/backend-day/07-layers $

© Ashan Dimantha

Back to the restaurant: who does what?

Restaurant	Layer	Its one job
Waiter	`Controller`	Takes the order, serves the plate. Never cooks.
Chef	`Service`	Knows the recipes and the rules: “no nuts for table 4”.
Pantry keeper	`Repository`	Knows where every ingredient lives. Fetches and stores.
Pantry	`Database`	Where the ingredients actually sit.



© Ashan Dimantha

How does a request travel through the layers?

Rule: each layer only talks to the one below. Nothing skips a layer.

© Ashan Dimantha

What runs before and after every request? Interceptors

Like airport checkpoints: every passenger passes them on the way in and out, whatever the destination.

request → log + time → check login → controller → service → repository → errors → response

```
# in Express, an interceptor is middleware
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () =>
    console.log(req.method, req.url, res.statusCode, Date.now() -
start, 'ms'));
  next();  // let the request continue
});
```

Cross-cutting jobs every route needs

Written **once**, not copied into 20 controllers

Can stop a request early: 401, 429…

© Ashan Dimantha

What does a controller do? Speak HTTP.

ROUTE HANDLING
Take one route's request, return its response

EXCEPTION HANDLING
When something fails, the client gets the right status code

```
# controllers/orderController.js
const orderService = require('../services/orderService');
async function create(req, res) {
  const order = await orderService.placeOrder(req.user.id, req.body.items);
  res.status(201).json(order);
}
module.exports = { create };
```

✓ **Does:** read the request, call a service, pick the status code

⚠ **Doesn't:** business rules or database queries

~/backend-day/07-layers $

© Ashan Dimantha

What does a service do? Make decisions.

BUSINESS LOGIC, SIMPLY

The shop's own rules, which would be true even without a website: "an order can't be empty", "gold members get 10% off", "don't sell what's out of stock".

```
# services/orderService.js
const repo = require('../repositories/orderRepository');
const pricing = require('./pricing');
async function placeOrder(userId, items) {
  if (!items?.length) throw new ValidationError('Order is empty');
  const total = await pricing.totalFor(userId, items);
  return repo.insert({ userId, items, total });
}
```

✓ **Does:** rules, calculations, orchestration

⚠ **Doesn't:** know about req, res or Mongo

~/backend-day/07-layers $

© Ashan Dimantha

What does a repository do? Fetch and store.

DATA ACCESS, SIMPLY

The only part that knows where and how data is stored. Everyone else just asks: “save this order”, “find book 42”.

```
# repositories/orderRepository.js
const Order = require('../models/order');
async function insert(data) {
  const doc = await Order.create(data);
  return { id: doc._id.toString(), total: doc.total, items: doc.items };
}
module.exports = { insert };
```

✓ **Does:** queries, and maps DB documents to plain objects

⚠ **Doesn’t:** decide anything about the business

~/backend-day/07-layers $

© Ashan Dimantha

ACTIVITY 07 · Which layer? · 3 min

Waiter, chef, or pantry keeper?

`res.status(404).json(…)`	Controller
`if (stock < qty) throw new OutOfStockError()`	Service
`Book.find({ author }).sort({ year: -1 })`	Repository
`const { title } = req.body`	Controller
`if (user.tier === 'gold') total *= 0.9`	Service
`return { id: doc._id.toString(), … }`	Repository

Point at a corner of the room: waiter · chef · pantry

© Ashan Dimantha

When are layers worth it, and when are they overkill?

Worth it

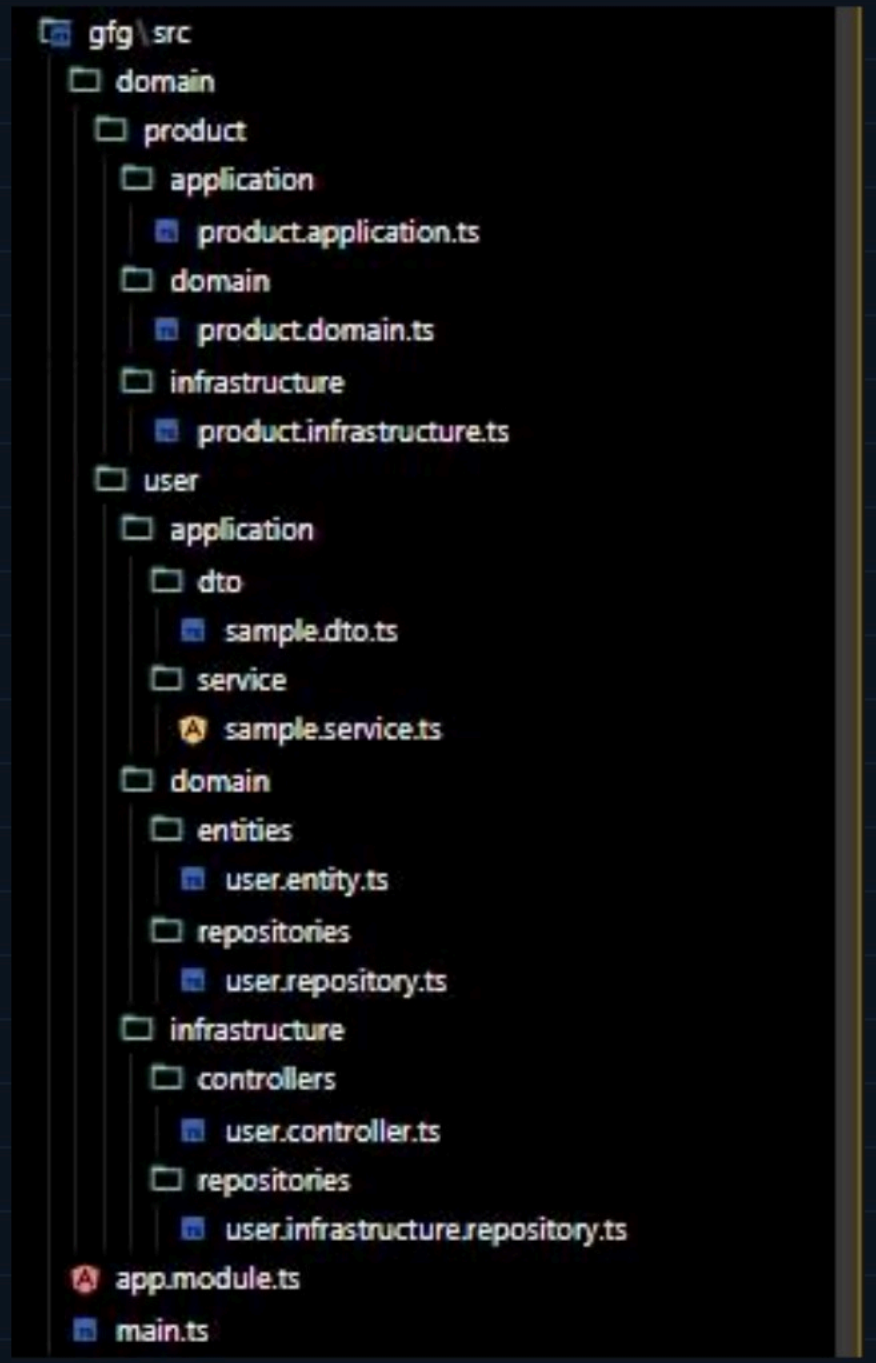
- Several endpoints share rules
- A team owns it for years
- You want fast unit tests

Overkill

- A one-off script
- A hackathon prototype
- Layers that only pass data through

`src/ → routes/ · controllers/ · services/ · repositories/ · models/`

© Ashan Dimantha



© Ashan Dimantha

ACTIVITY 07 · Mini-quiz · 3 min

Layers: quick check

1 · “Refunds only within 30 days.” Which layer?	Service
2 · A controller imports mongoose. Smell or fine?	Smell: it skips two layers
3 · Switching MongoDB to PostgreSQL. What changes?	Repositories and models

Shout-outs welcome

© Ashan Dimantha

Step 7 · Can we move the routes into their own file?

```
# new file: routes/books.js
const express = require('express');
const router = express.Router();
const books = [ … ];  // moved from app.js
router.get('/', (req, res) ⇒ res.json(books));
router.get('/:id', …);  // same body as before
router.post('/', …);    // same body as before
module.exports = router;
```

```
# app.js · replaces the old routes
app.use('/books',
  require('./routes/books'));
```

app.get becomes **router.get**

'/books/:id' becomes **'/:id'**: app.use adds /books

Checkpoint: nothing changes from outside. GET, GET one and POST answer exactly as before.

~/backend-day/lab1 $

© Ashan Dimantha

ACTIVITY 08 · Databases · why we need one

The server stopped. Where did our books go?

IN MEMORY · const books = [ ]

lives inside the running server

Dune

Neuromancer

Your book

[ ] .. all gone

server restarts

IN A DATABASE

lives on disk, outside the server

- Dune
- Neuromancer
- Your book

REMEMBER
Data survives restarts, crashes and deploys

SHARE
Two servers see the same books

FIND FAST
Search millions of rows with indexes

STAY SAFE
Many users at once, plus backups

Remember the Lab 1 twist? This is why.

© Ashan Dimantha

ACTIVITY 08 · Databases · poll

13:15 → 13:35 · 20 min

Where does your team's data live?

A	Relational: Postgres, MySQL, SQL Server	__%
B	Document: MongoDB and friends	__%
C	A mix, plus a cache like Redis	__%
D	A spreadsheet someone emails around	__%

Vote: [poll link or QR code] · no judgement on D

© Ashan Dimantha

Why not just save our books to a JSON file?

Two writes at once

Both read, both write, one update vanishes.

One in a million

Finding a record means reading the whole file.

Crash halfway

A half-written file is a corrupted file.

Many servers

Three API instances, three different files.

© Ashan Dimantha

Linked spreadsheet tabs, or a folder of complete cards?

RELATIONAL = LINKED TABS

books	title	author_id
1	Dune	7

authors	name
7	Frank Herbert

Split into tables, joined back by ids.

DOCUMENT = COMPLETE CARDS

```
{ "_id": "…a1",
  "title": "Dune",
  "author": { "name": "Frank Herbert" },
  "tags": ["sci-fi", "classic"] }
```

Everything you need, in one card.

© Ashan Dimantha

How do they differ in practice?

	Relational	Document (MongoDB)
Shape	Tables and rows	JSON-like documents
Schema	Defined up front	Flexible, optionally validated
Relations	Foreign keys and joins	Embed, or reference by id
Transactions	Multi-row, the classic strength	Single-doc atomic; multi-doc supported
Queries	SQL	Query API and aggregation pipeline
Scaling out	Replicas; sharding is extra work	Replica sets and built-in sharding

© Ashan Dimantha

How does a library find one book in a million?

WHAT IS AN INDEX? A **sorted list** of one field's values, each pointing to where its document lives, so the database **jumps straight there** instead of reading everything. Like the index at the back of a book.

NO INDEX = COLLECTION SCAN

Walk every shelf, check every spine.

INDEX = THE CATALOGUE

Look up the card, walk straight to the shelf.

```
bookSchema.index({ author: 1, year: -1 }); // Mongoose
db.books.createIndex({ author: 1, year: -1 }) // mongosh
```

index on author (sorted A→Z): `Asimov → #17` `Herbert → #42` `Le Guin → #8` `Tolkien → #301`

The price: **every write also updates the catalogue**, and the catalogue takes space.

© Ashan Dimantha

When should I pick MongoDB?

Good fit

- Data is naturally one “card”
- Shapes evolve often
- You read the whole object at once

Lean relational

- Many-to-many everywhere
- Money and ledgers
- Ad-hoc reporting in SQL

Design for how you **read** the data, not just how you store it.

© Ashan Dimantha

What are the only four things an app does with data?

C

Create

Write a new card and file it

Sign up · post a photo

R

Read

Look a card up, or flip through the drawer

Open your feed · search

U

Update

Cross something out, write the new value

Change your photo

D

Delete

Take the card out of the drawer

Remove a post

Every screen you've ever used is some mix of these four. That's **CRUD**.

© Ashan Dimantha

How does one CRUD job travel through the stack?

CRUD	HTTP	Success	Mongoose (Lab 2)	SQL
Create	POST /books	201	Book.create()	INSERT
Read	GET /books(/:id)	200	find() · findById()	SELECT
Update	PATCH /books/:id	200	findByIdAndUpdate()	UPDATE
Delete	DELETE /books/:id	204	findByIdAndDelete()	DELETE

Read one that doesn't exist? **404**. Same for update and delete of a missing id.

© Ashan Dimantha

ACTIVITY 08 · Shout it out · 1 min

C, R, U or D? Shout the letter

1 · You add a book to your wishlist	C · POST
2 · You search for “Dune”	R · GET
3 · You fix a typo in your address	U · PATCH

Next: we build all four in Lab 2

© Ashan Dimantha

What is MongoDB?

A **document database**: it stores data as JSON-like documents instead of rows in tables.

```
# one document in the books collection
{
  "_id": "6650f1a2…",
  "title": "Dune",
  "year": 1965,
  "tags": ["sci-fi", "classic"],
  "author": { "name": "Frank Herbert" }
}
```

SPEAKS JSON

Same shape as our API, no translation

FLEXIBLE

Lists and nested objects live inside one document

COLLECTIONS

Groups of documents, like tables without fixed columns

RUNS ANYWHERE

On your laptop, in Docker, or in the cloud (Atlas)

mongoDB®

database → collections → documents · and in Node we talk to it with Mongoose

© Ashan Dimantha

Why MongoDB for our Books API?

```
# SQL
SELECT * FROM books
WHERE year > 1960;
```

```
# MongoDB, from JavaScript
Book.find({
  year: { $gt: 1960 } });
```

- **JSON all the way:** request, code and database share one shape
- **Start in minutes:** no tables to design first, free cloud tier
- **Popular:** the M in MERN, huge community and docs
- **Grows with you:** copies for safety, spreads across servers

Honest note: not always the best choice. Money and heavily linked data often fit SQL better.

Same question, two languages: SQL speaks its own; MongoDB speaks JavaScript objects

© Ashan Dimantha

LAB 09 · Lab 2 · Express + MongoDB

13:35 → 14:05 · 30 min

Lab 2: let's give our books a real home

First time with a database? Perfect. We change Lab 1 in small steps, and you can run the app after every one.

ONLY 3 NEW IDEAS

- Connect to MongoDB
- Describe a book (a model)
- Save and find books

ONLY 2 NEW FILES

- models/book.js
- repositories/bookRepository.js
- ...plus small edits to app.js

NOTHING TO INSTALL

- No Docker needed
- We hand you a ready database link
- Same pair as Lab 1

1 warm-up · 7 small steps · a checkpoint after each

~/backend-day/lab2 $

VERIFY: Mongoose major version

© Ashan Dimantha

Warm-up · What does MongoDB store, in plain words?

DATABASE	COLLECTION	DOCUMENT
A filing cabinet	One drawer	One card in the drawer
books-pair07	books	one book

```
{ "_id": "6650f1…",
  "title": "Dune",
  "author": "Frank Herbert",
  "year": 1965 }
```

A document looks just like the JSON you sent in Lab 1. MongoDB adds the _id for you.

© Ashan Dimantha

CONCEPT 09 · Lab 2 · Mongoose

What is Mongoose?

A Node.js library that sits between our code and MongoDB: it turns JavaScript objects into documents, and back.

SCHEMA
the blank form: fields and rules

MODEL
the tool to save and find (Book)

DOCUMENT
one filled-in form (one book)

Officially: an ODM, Object Data Modeling library for MongoDB

© Ashan Dimantha

CONCEPT 09 · Lab 2 · why Mongoose

Why not just talk to MongoDB directly?

Spot the typo: someone wrote **titel** instead of title.

```
# plain driver
db.collection('books')
  .insertOne({ titel: 'Dune' });
→ saved. A book with no title.
```

```
# Mongoose
Book.create({ titel: 'Dune' });

→ ValidationError: title is required
```

Rules: required fields, types, min/max, defaults

One place for the shape: the schema documents it

Shortcuts: findById, timestamps, ids made easy

Trade-off: one more layer to learn, a little slower

MongoDB is flexible; Mongoose adds the guard rails

© Ashan Dimantha

Step 1 · Get ready: one install, one secret

```
# in your Lab 1 folder (dotenv is already there)
$ npm install mongoose
```

```
# new file: .env (paste the link we gave you)
MONGO_URL=mongodb+srv://…/books-pair07
```

MONGOOSE

A helper that lets JavaScript talk to MongoDB

.ENV

Keeps the secret link out of your code. Never commit it.

Checkpoint: package.json lists mongoose, and .env sits next to app.js.

~/backend-day/lab2 $

© Ashan Dimantha

Why not just paste the database link into the code?

✗ HARD-CODED

```
mongoose.connect(
  'mongodb+srv://ashan:Pa55word@…'
);
```

The password is now in git, forever

✓ FROM THE ENVIRONMENT

```
mongoose.connect(
  process.env.MONGO_URL
);
```

The code only says *which* setting it needs

Safe: secrets never reach GitHub

Flexible: laptop, test and AWS use different values

Easy: change a value without changing code

~/backend-day/lab2 $

© Ashan Dimantha

LAB 09 · Lab 2 · MongoDB Atlas setup ~5 min

How do we get our own free MongoDB link?

1 Sign up
mongodb.com/atlas · use Google or email

2 Create a free cluster
Free tier · AWS · nearest region (e.g. Mumbai)

3 Add a database user
Username + password · save the password!

4 Allow your IP
Network Access → Add current IP address

5 Copy the link
Connect → Drivers → Node.js

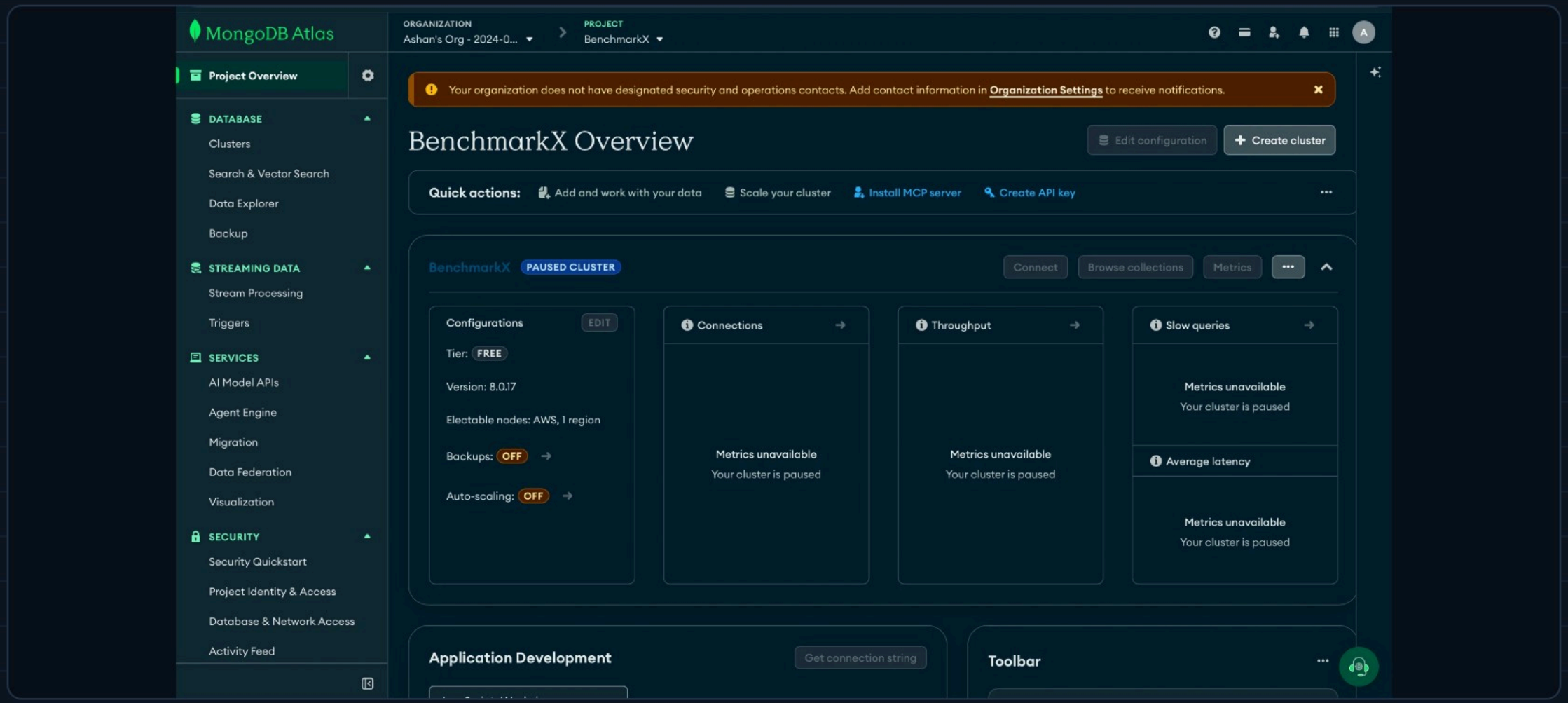
6 Paste into .env
Replace the password · add /books

```
# .env · your link, with your user, password and database name
MONGO_URL=mongodb+srv://ashan:MyPass123@cluster0.abcde.mongodb.net/books?retryWrites=true
                        user · password · your cluster's address · database name
```

Password with @ # or : in it? Pick one without, it saves a headache

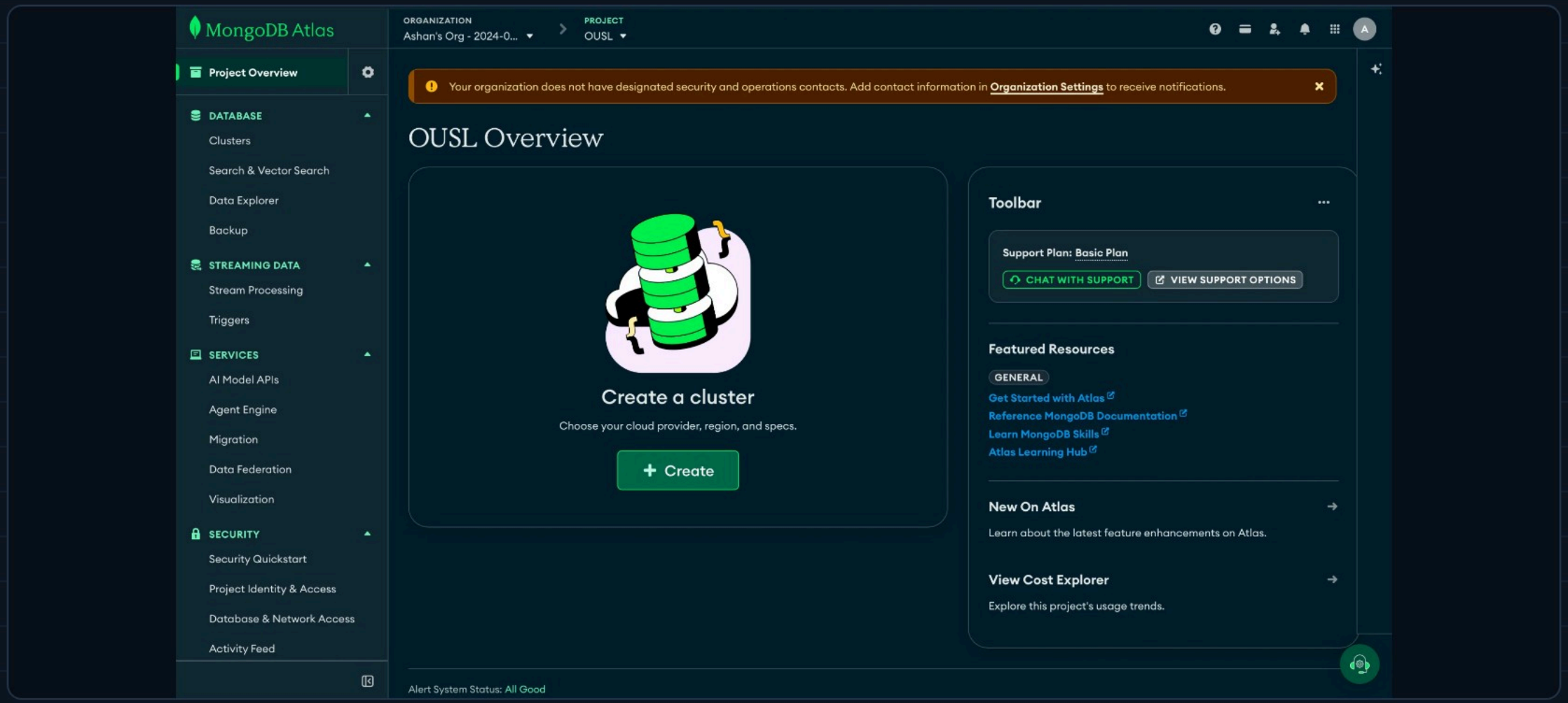
VERIFY: Atlas screens and free tier

© Ashan Dimantha



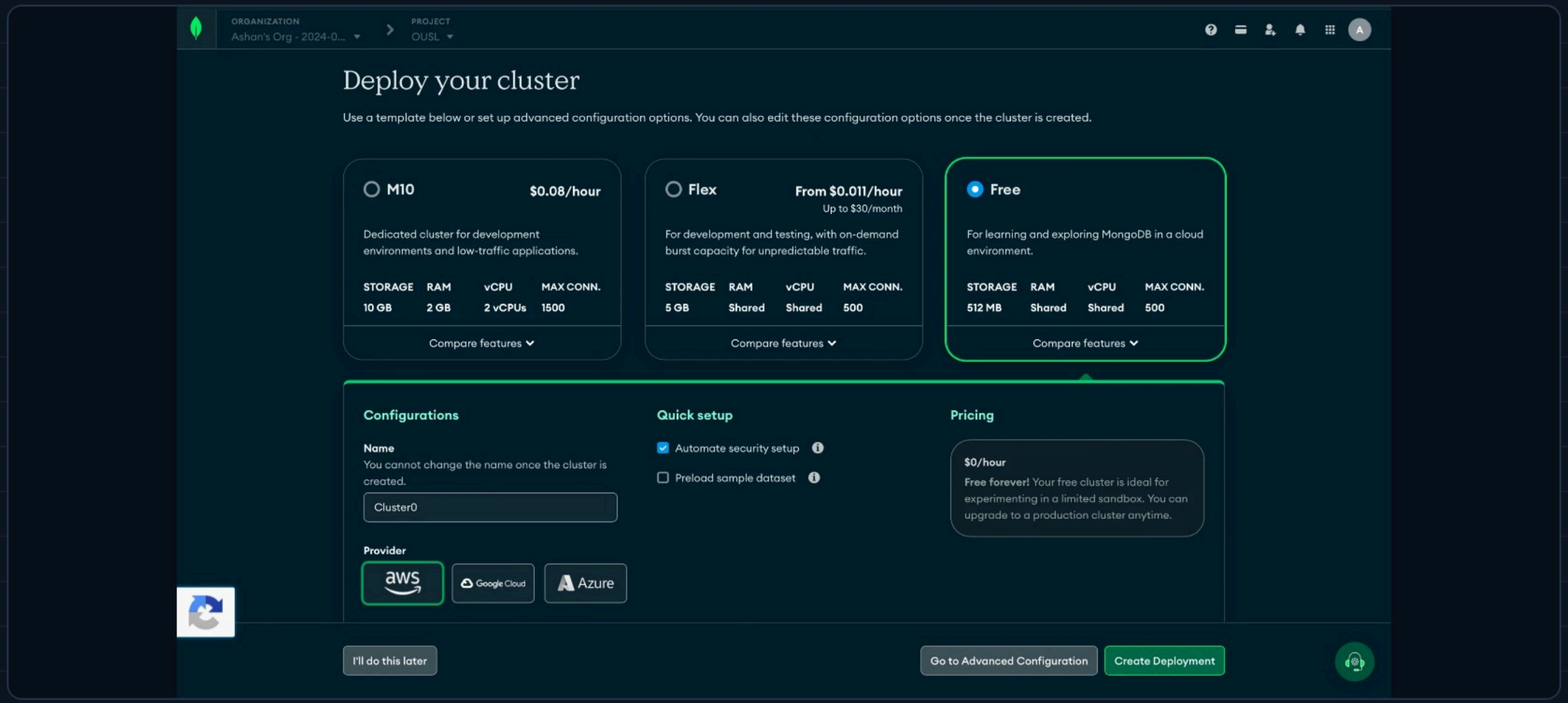
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



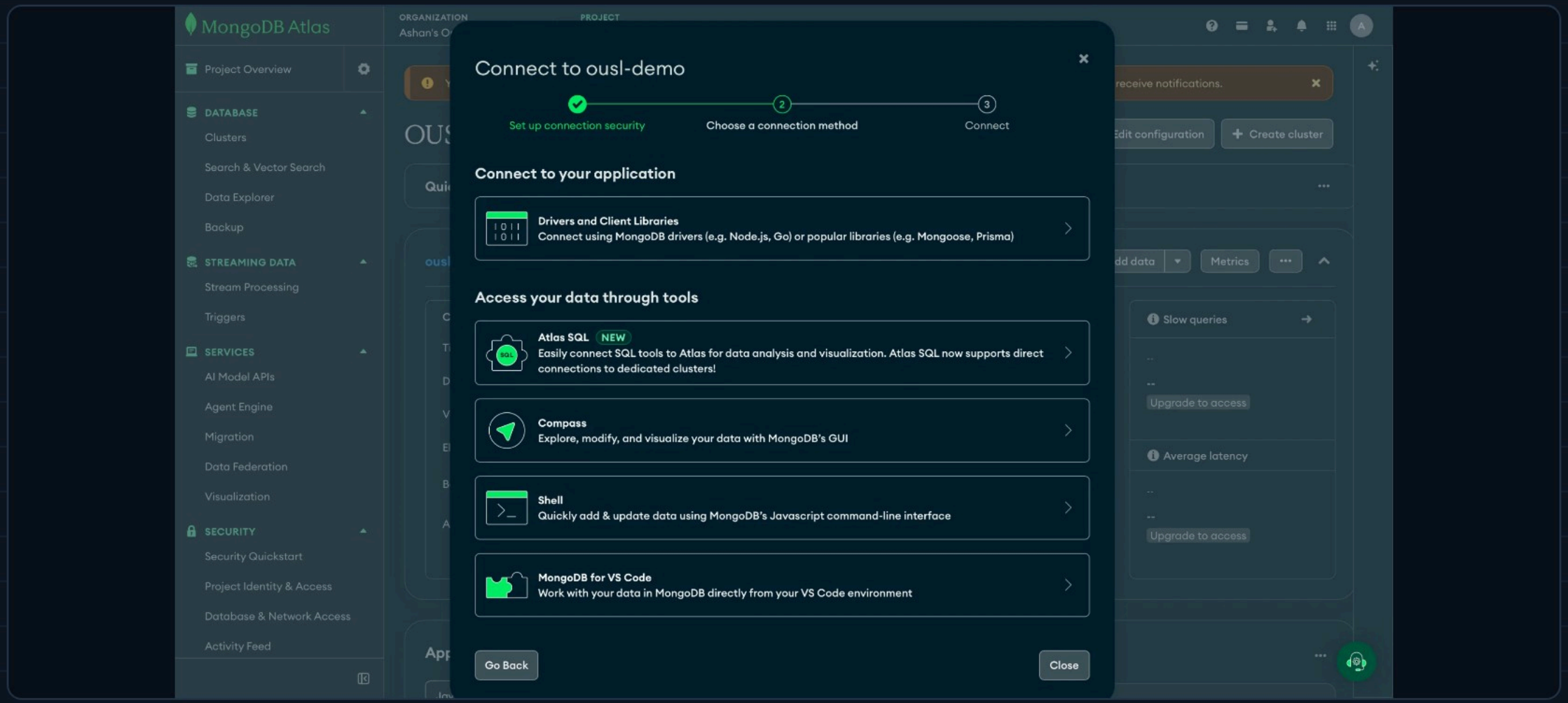
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



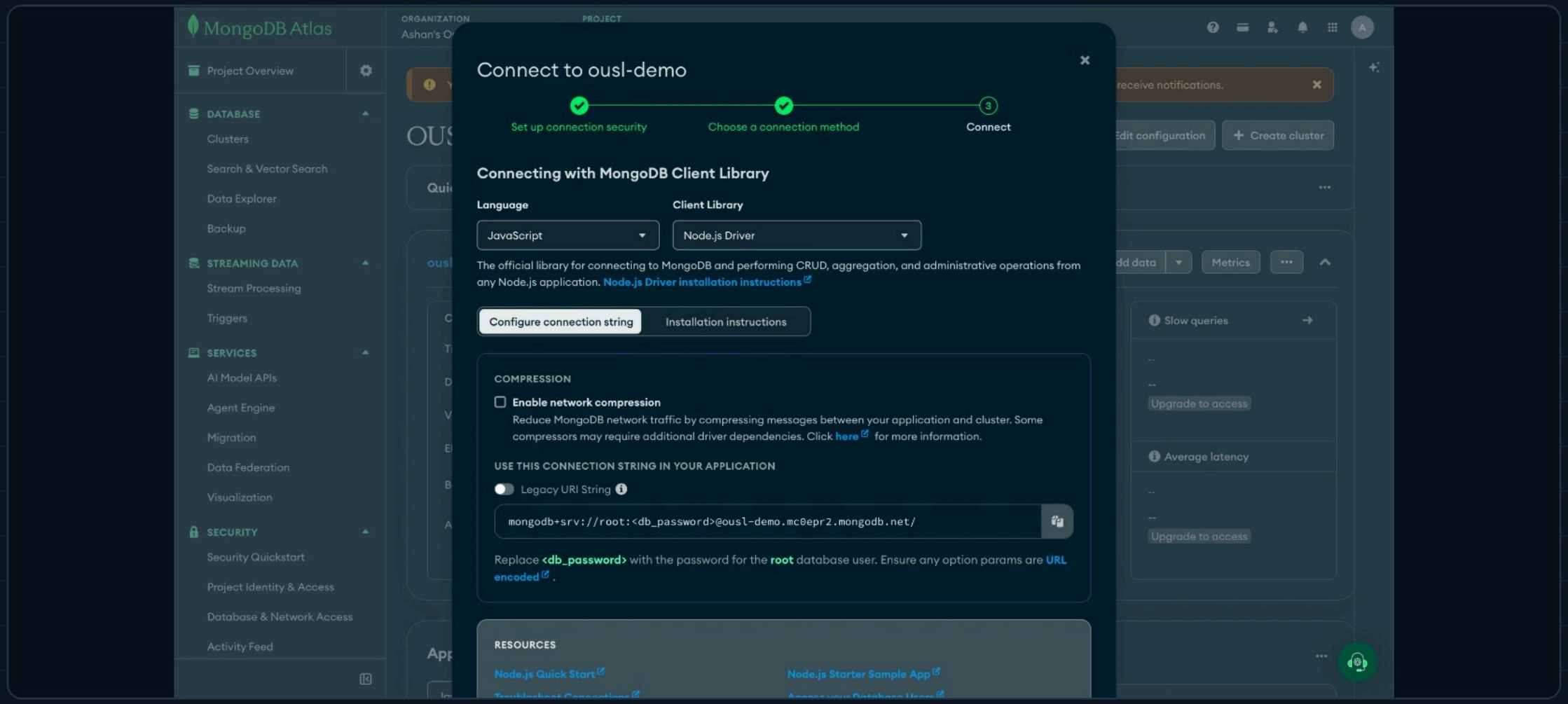
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



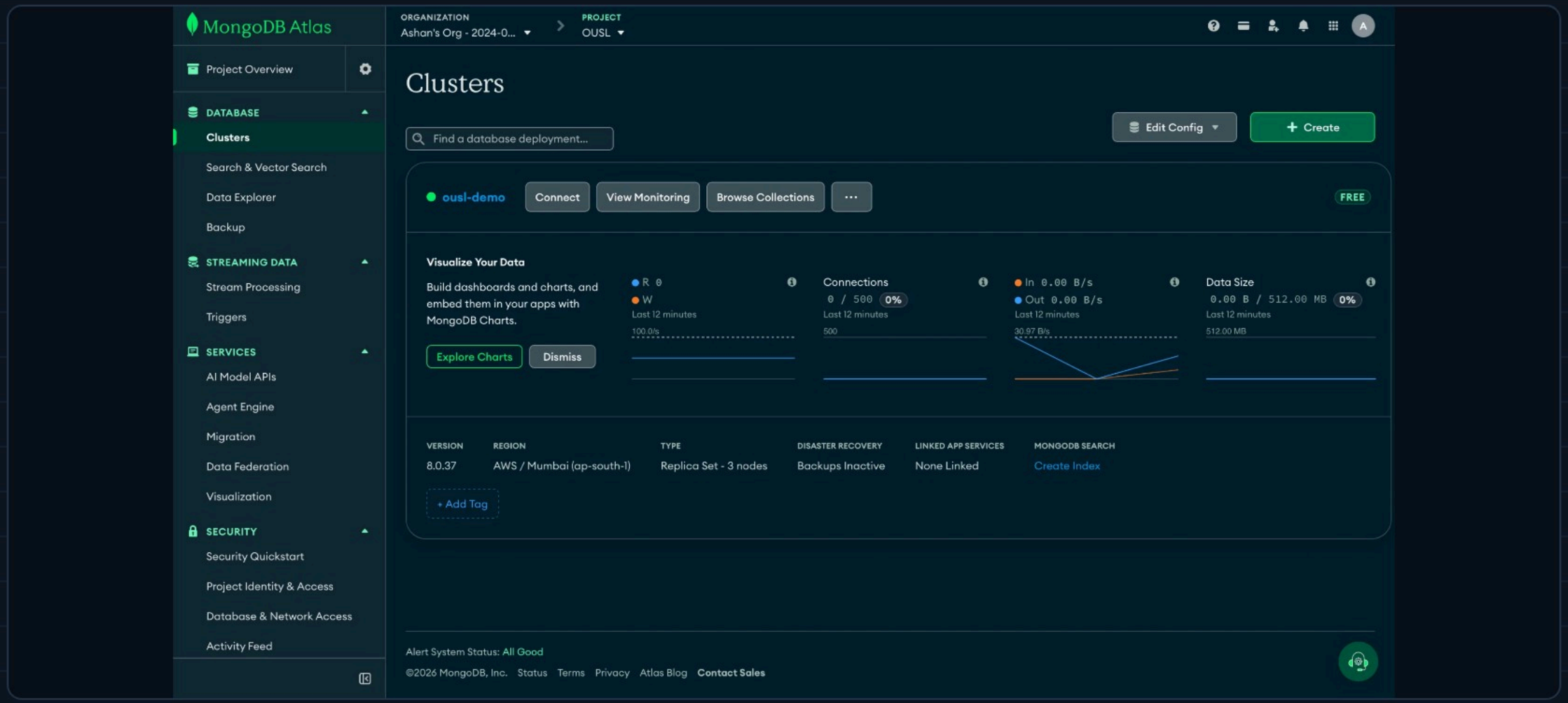
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



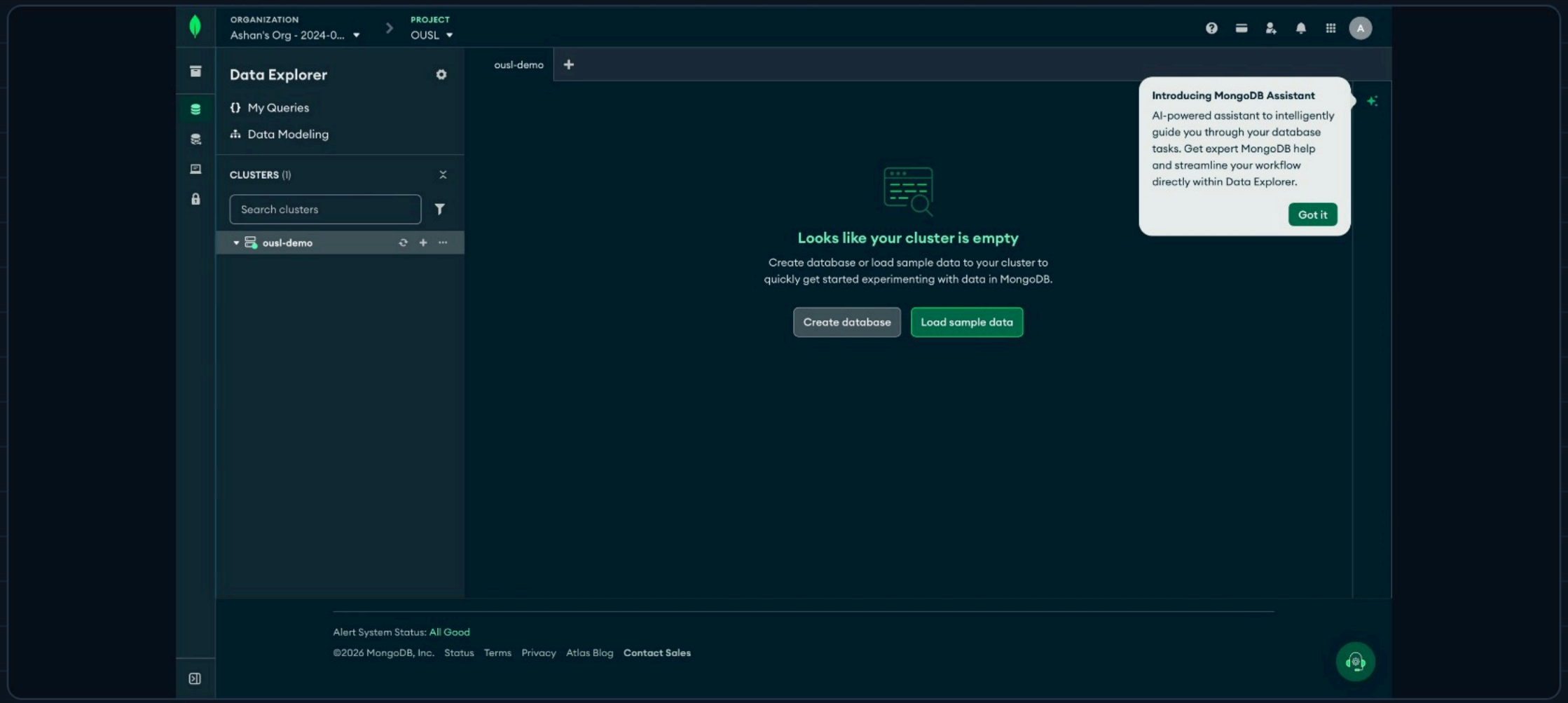
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



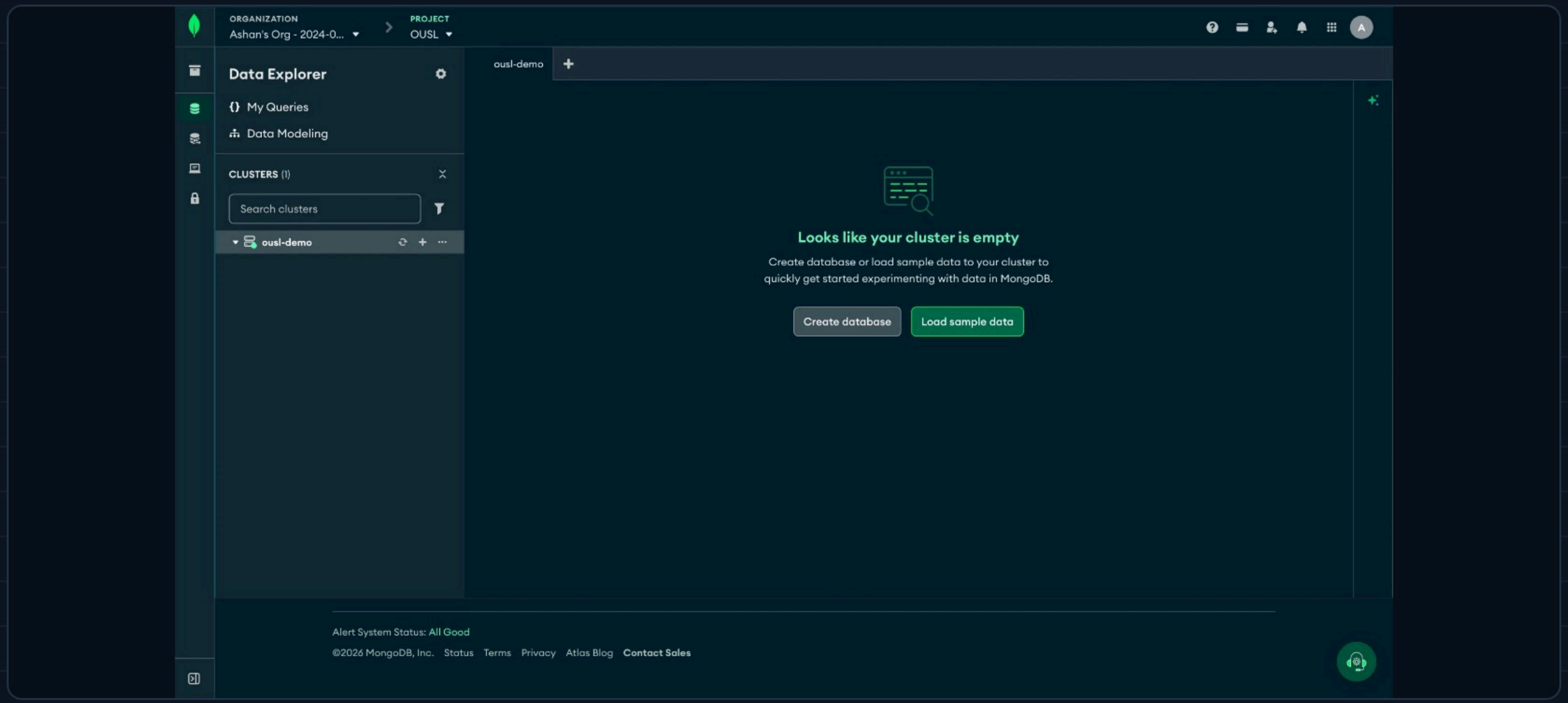
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



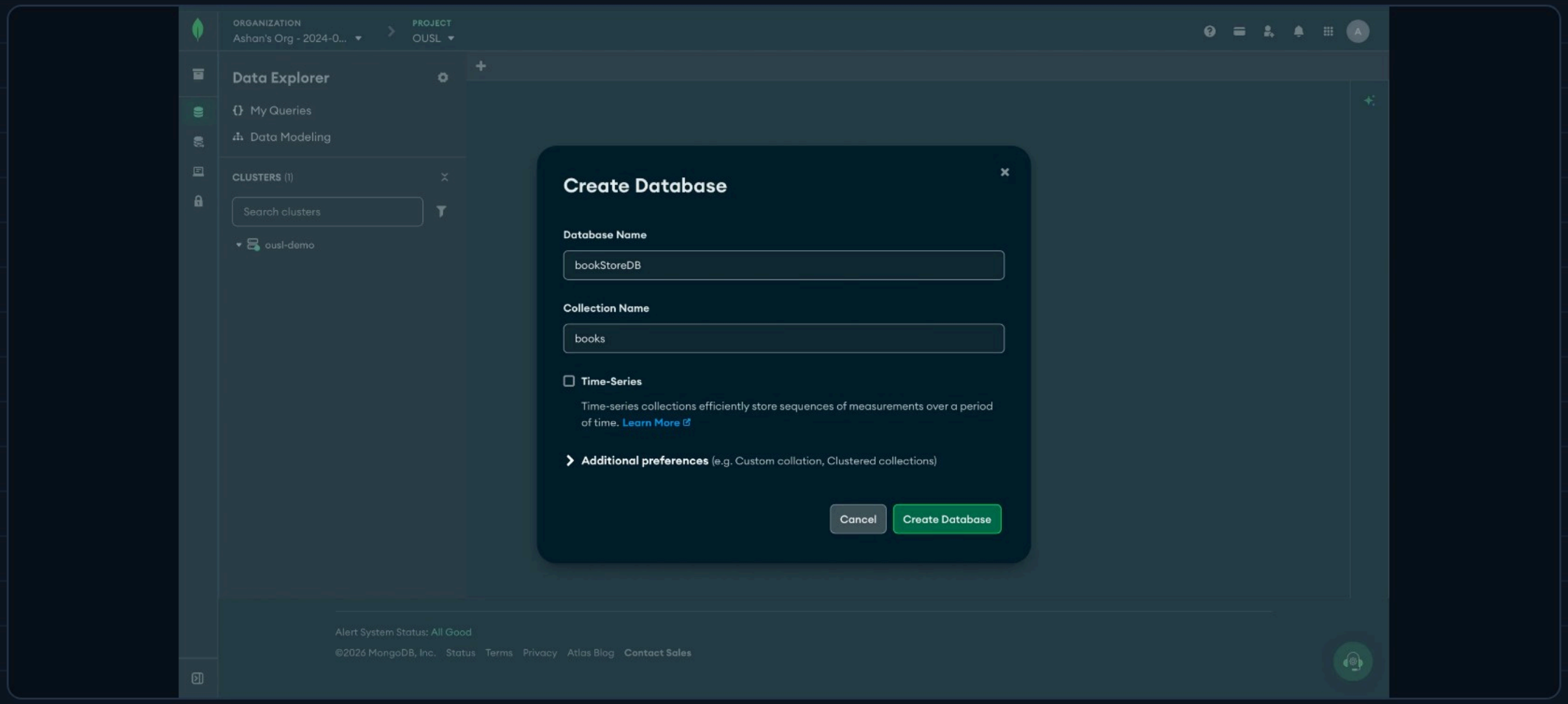
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



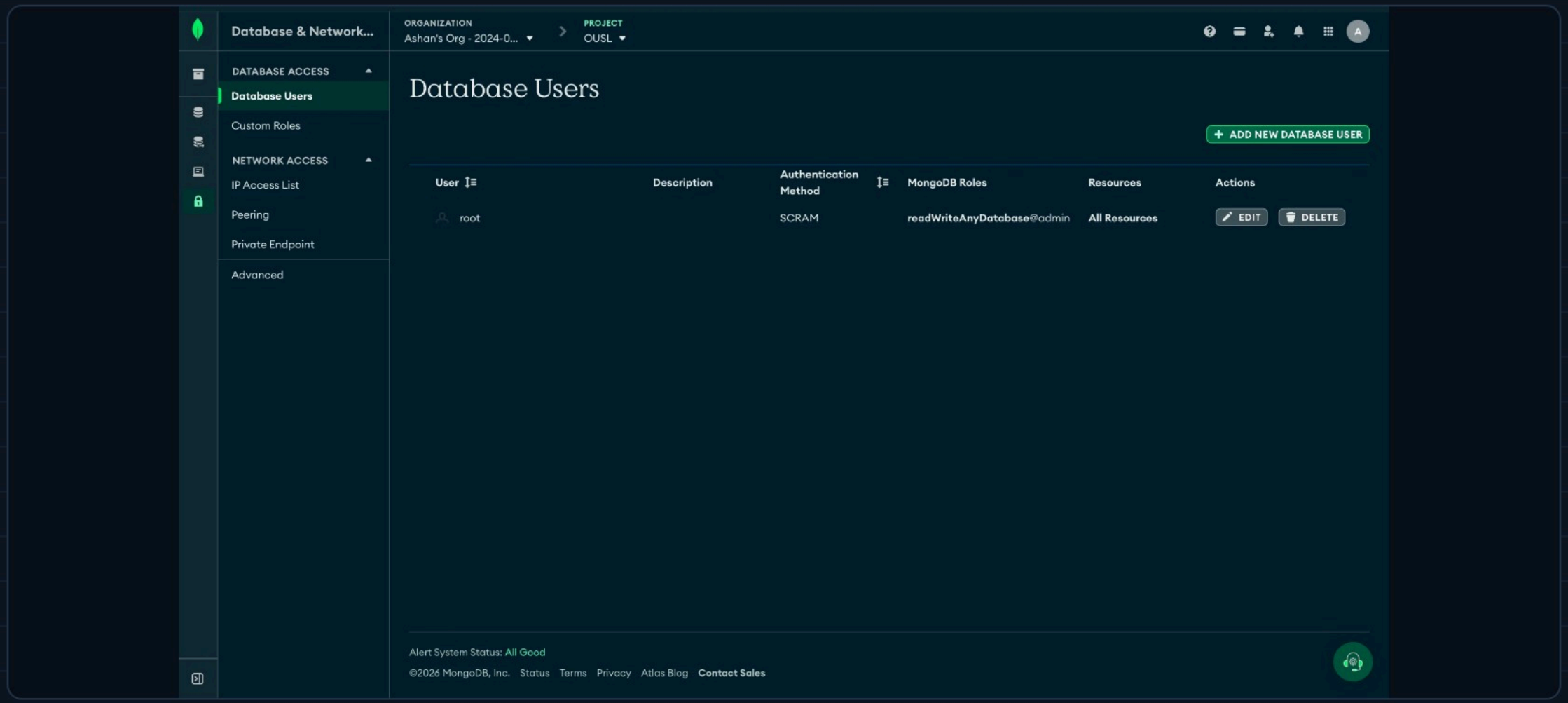
~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha



~/backend-day/lab2 $ follow along on your laptop

© Ashan Dimantha

Step 2 · Can our app reach the database?

```
# app.js · add these 3 lines at the very top
require('dotenv').config();
const mongoose = require('mongoose');
mongoose.connect(process.env.MONGO_URL).then(() ⇒ console.log('MongoDB connected'));
const express = require('express');
// …the rest of your Lab 1 code, unchanged
```

Read it aloud: “load my secrets, load the helper, connect using the link.”

Checkpoint: run npm run dev and see “MongoDB connected”.

~/backend-day/lab2 $

© Ashan Dimantha

Step 3 · What does a book card look like?

```
# new file: models/book.js
const mongoose = require('mongoose');
const bookSchema = new mongoose.Schema({
  title: { type: String, required: true },
  author: String,
  year: Number,
});
module.exports = mongoose.model('Book', bookSchema);
```

SCHEMA

The blank card: which fields, which types, which are required

MODEL

The drawer of book cards we save into and search

~/backend-day/lab2 $

© Ashan Dimantha

Step 4 · Who saves and finds books for us?

```
# new file: repositories/bookRepository.js
const Book = require('../models/book');
module.exports = {
  create: (data) ⇒ Book.create(data),
  findAll: () ⇒ Book.find(),
};
```

This is the repository from this morning. The rest of the app just says “create” or “find all” and never touches MongoDB directly.

We start with two jobs. We add three more in steps 6 and 7.

© Ashan Dimantha

Step 5 · Create and list books

```
# app.js · write these right after const app = express();
const repo = require('./repositories/bookRepository');
app.use(express.json());
app.post('/books', async (req, res) ⇒ {
  const book = await repo.create(req.body);
  res.status(201).json(book);
});
app.get('/books', async (req, res) ⇒ res.json(await repo.findAll()));
```

ASYNC / AWAIT

The database answers later, so we wait for it

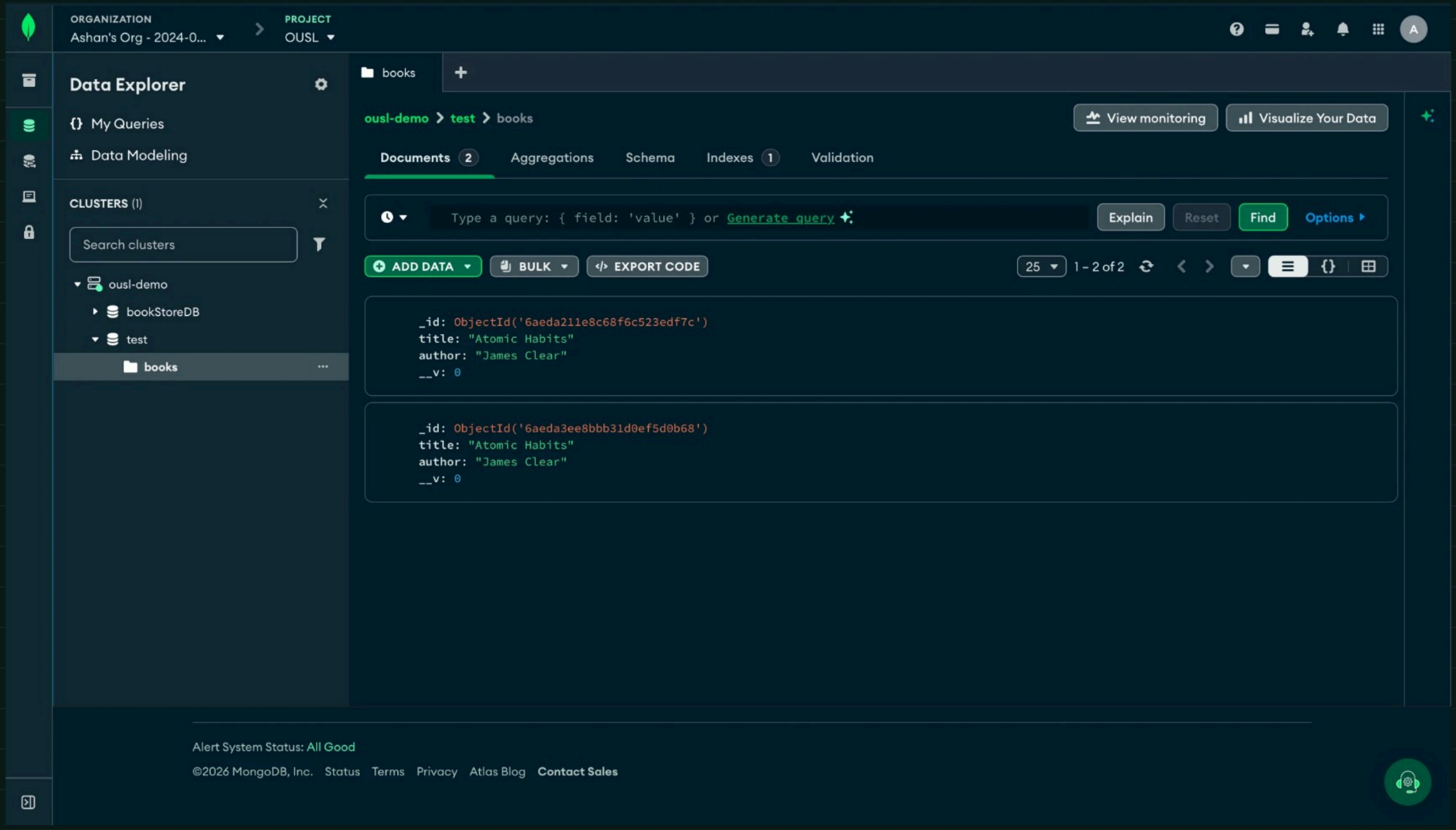
START EMPTY

No sample array: every book comes from a POST

Checkpoint: POST two books in Postman, GET /books, restart, GET again. Still there!

~/backend-day/lab2 $

© Ashan Dimantha



© Ashan Dimantha

LAB 09 · Lab 2 · where do the books land?

Where exactly will our books be saved?

```
# .env · add the database name right after .net/
MONGO_URL=mongodb+srv://ashan:MyPass123@cluster0.abcde.mongodb.net/?retryWrites=true
MONGO_URL=mongodb+srv://ashan:MyPass123@cluster0.abcde.mongodb.net/bookstore?retryWrites=true
```

DATABASE · from the URL

/bookstore → the **bookstore** database

Leave it out and it goes to “test”

COLLECTION · from the model

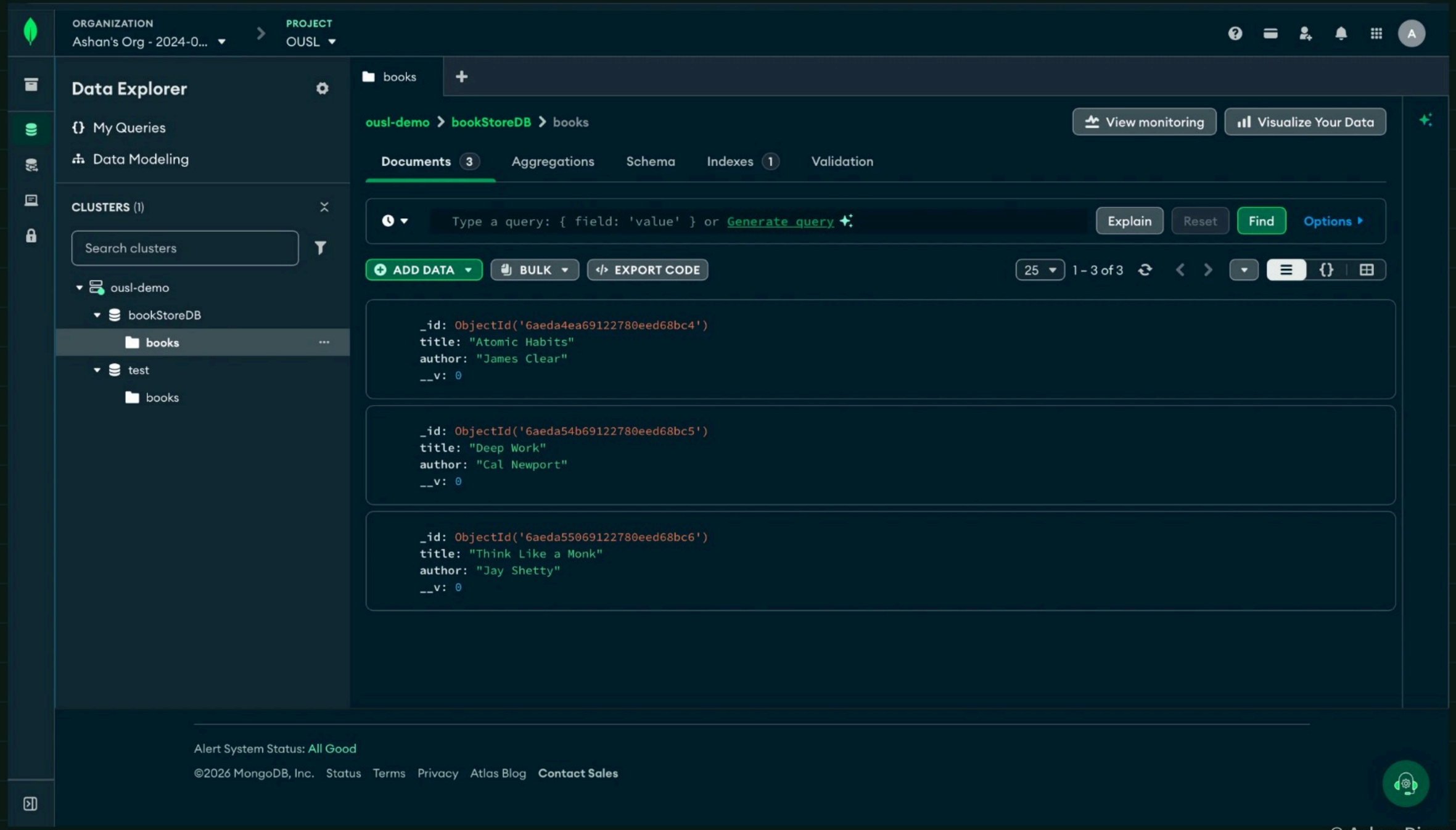
`model('Book', bookSchema, 'books')`

3rd argument = the exact collection name

`cluster0 → bookstore → books → { "title": "Dune" }`

Checkpoint: after your first POST, Atlas → Browse Collections shows bookstore › books

© Ashan Dimantha



© Ashan Dimantha

Step 6 · Can we find one book?

```
# bookRepository.js · add one line inside module.exports
  findById: (id) ⇒ Book.findById(id),
```

```
# app.js · replaces the old GET /books/:id
app.get('/books/:id', async (req, res) ⇒ {
  const book = await repo.findById(req.params.id);
  if (!book) return res.status(404).json({ error: 'Book not found' });
  res.json(book);
});
```

Checkpoint: copy an _id from GET /books, then GET /books/that-id returns just that book.

~/backend-day/lab2 $

© Ashan Dimantha

CONCEPT 09 · Lab 2 · when things go wrong

What is exception handling?

Catching errors **on purpose**, so the app answers with a clear message instead of crashing or leaking details.

SOMETHING BREAKS
Bad id, missing title, database down

IT'S THROWN
An error jumps out of the normal path

WE CATCH IT
One place turns it into the right status code

```
# app.js · last, after every route
app.use((err, req, res, next) ⇒ {
  if (err.name === 'ValidationError')
    return res.status(400).json({ error: err.message });
  if (err.name === 'CastError')
    return res.status(400).json({ error: 'Invalid id' });
  console.error(err);
  res.status(500).json({ error: 'Something went wrong' });
});
```

Try it: GET /books/abc → 400 Invalid id, not a 500

© Ashan Dimantha

A simple example: try / catch

✗ WITHOUT · hope nothing breaks

```
app.get('/books/:id', async (req, res) => {
  const book = await repo.findById(req.params.id);
  res.json(book);
});
```

GET /books/abc
→ 500 and a scary CastError

✓ WITH · a plan B

```
app.get('/books/:id', async (req, res) => {
  try {
    const book = await repo.findById(req.params.id);
    if (!book)
      return res.status(404).json({ error: 'Not found' });
    res.json(book);
  } catch (err) {
    res.status(400).json({ error: 'Invalid id' });
  }
});
```

GET /books/abc
→ 400 Invalid id

GET a deleted id
→ 404 Not found

try = the risky part · catch = plan B when it fails

© Ashan Dimantha

Step 7 · Can we change and delete a book?

```
# bookRepository.js · two more lines inside module.exports
  update: (id, data) ⇒ Book.findByIdAndUpdate(id, data, { new: true }),
  remove: (id) ⇒ Book.findByIdAndDelete(id),
```

```
# app.js · two more routes
app.patch('/books/:id', async (req, res) ⇒ res.json(await repo.update(req.params.id, req.body)));
app.delete('/books/:id', async (req, res) ⇒ { await repo.remove(req.params.id);
res.status(204).end(); });
```

That’s all of CRUD: Create · Read · Update · Delete, every one through the repository.

© Ashan Dimantha

Does everything work? Tick them off

Try	Expect
POST /books {"title":"Dune","year":1965}	201 + an _id
GET /books	200, Dune is listed
GET /books/that-id	200, just Dune
PATCH /books/that-id {"author":"Herbert"}	200, author added
DELETE /books/that-id	204, then it’s gone
Restart the server, then GET /books	Your books are still there

Use curl -i (add -H "Content-Type: application/json" for bodies) or any HTTP client.

~/backend-day/lab2 $

© Ashan Dimantha

LAB 09 · Lab 2 · help desk

Stuck? Find your error here

You see	Try this
“MongoDB connected” never appears	Check .env: no spaces, no quotes, the whole link on one line.
`uri must be a string`	.env isn’t being read: it must sit next to app.js, named exactly .env
`bad auth`	Ask us for a fresh link; the password part may be mistyped.
`Cannot find module '../models/book'`	Check the folder and file names: models/book.js
GET /books still shows old books	Delete the old books array and old routes from Lab 1.
A request never answers	Missing await, or the new routes sit below the 404 handler.

Red sticky note = help on the way

© Ashan Dimantha

ACTIVITY 09 · Lab 2 debrief · 4 min

What did the layers buy us?

1 · Which files would change to move from MongoDB to Postgres?	Repository, model, connect lines
2 · Why did your books survive the restart this time?	They live in the database now
3 · Where would “no duplicate titles” go, as a next level?	A service, the chef

Fist of five again: how confident now?

© Ashan Dimantha

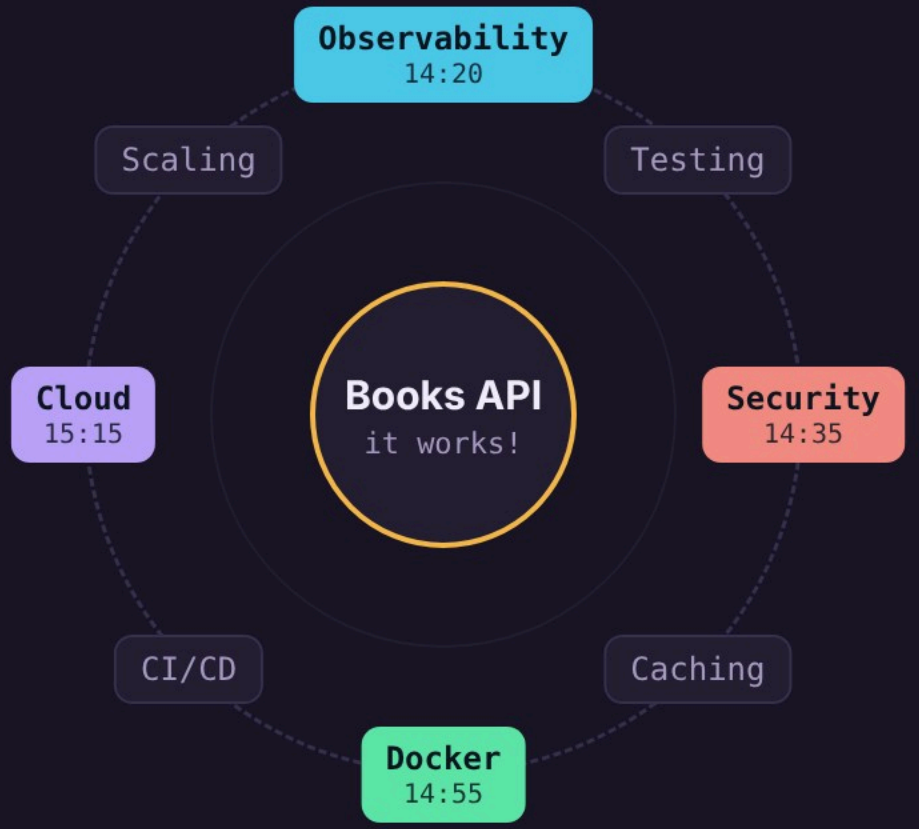
NEXT UP 10 · Beyond CRUD · the afternoon

Beyond CRUD

The other concerns of back-end engineering

Our Books API works on your laptop. What else does it need before **real users** arrive?

- bright = this afternoon
- dim = your next steps after today



Working code is step one. Production is the rest.

© Ashan Dimantha

THEORY 10 · Observability · face a question

14:20 → 14:35 · 15 min

Users say “checkout is slow”. Where do you look first?

REVEAL · three signals

Metrics tell you something is wrong. **Traces** tell you where. **Logs** tell you why.

~/backend-day/10-observability $

© Ashan Dimantha

What are logs, metrics and traces, in real life?

Logs

A ship’s logbook

One entry per event, with details.
Answers “what exactly happened?”

Metrics

A car’s dashboard

Numbers over time: speed, fuel.
Answers “how much, how often?”

Traces

Parcel tracking

Every scan on one parcel’s journey.
Answers “where did the time go?”

© Ashan Dimantha

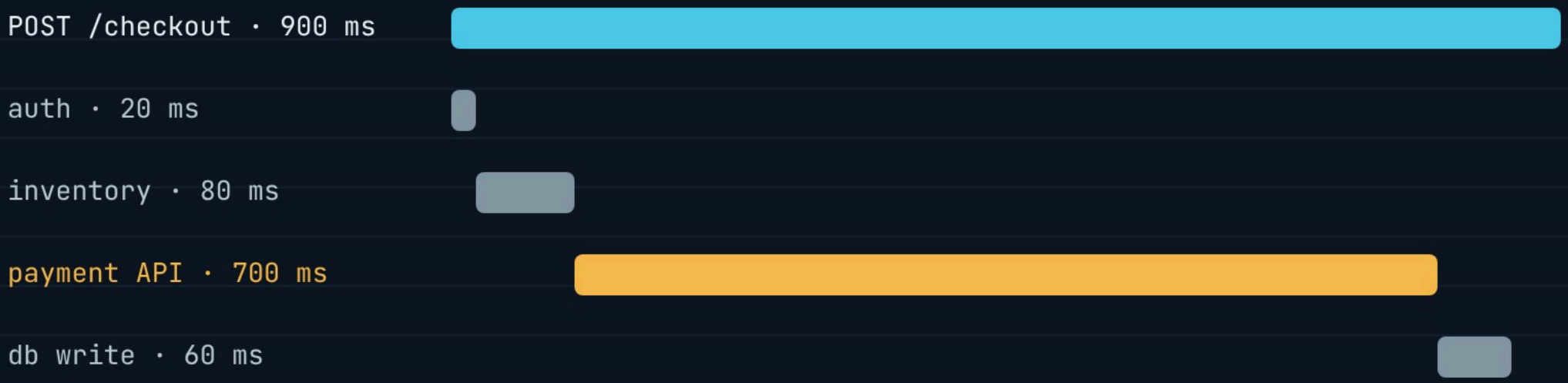
What makes a log line useful at 3 a.m.?

```
{
  "level": "error",
  "time": "2026-11-02T13:04:11Z",
  "requestId": "a1b2c3",
  "route": "POST /orders", "status": 502,
  "durationMs": 842,
  "msg": "payment provider timeout"
}
```

- JSON, not prose
- A request id on every line
- Levels: debug → error
- No passwords, tokens or card numbers

© Ashan Dimantha

Where did the 900 ms go?



One trace = one request's journey. Each bar is a **span**. The culprit: the payment API, not our code.

© Ashan Dimantha

Authentication vs authorization

Two different questions, always asked **in this order**.

1 · AUTHENTICATION · authN

“Who are you?”

Prove your identity: password, Google sign-in, a token, an API key

Fails → 401 Unauthorized

2 · AUTHORIZATION · authZ

“What may you do?”

Check permission: only admins delete books, you edit only your own review

Fails → 403 Forbidden

`DELETE /books/42` → `logged in?` → `admin?` → `delete the book · 204`

Airport: passport check = authN · boarding pass for THIS flight = authZ

© Ashan Dimantha

Who are you, and what may you do?

AUTHENTICATION · 401

Hotel reception checks your ID and hands you a key card.

Login → session cookie or token

AUTHORIZATION · 403

The key card opens your room, not every room.

Roles, ownership, permissions

```
router.delete('/:id', requireAuth, requireRole('admin'), c.remove);
```

Hash passwords (bcrypt, argon2). Prefer a proven identity provider over rolling your own.

© Ashan Dimantha

What is rate limiting?

A cap on **how many requests** one client may send **in a time window**.

limit: 5 requests per minute · one IP

200	200	200	200	200	429	429	429

429 Too Many Requests · Retry-After: 42 → try again later

PASSWORD GUESSING
1,000 login tries a minute

SCRAPERS · BUGGY LOOPS
One client slows it for everyone

SURPRISE BILLS
Every LLM call costs money

Count per key (IP, user or API key) · per window · reject the extra

© Ashan Dimantha

What stops one client from flooding us?

A club bouncer: only so many people per minute, and the rest wait outside.

```
const rateLimit = require('express-rate-limit');
app.use('/login', rateLimit({ windowMs: 15 * 60 * 1000, limit: 10 }));
```

- Over the limit → 429 + Retry-After
- Several servers → share counters (e.g. Redis), or limit at the gateway

© Ashan Dimantha

THEORY 12 · Docker · face a question

14:55 → 15:15 · 20 min

“It works on my machine.” So how do we ship your machine?

REVEAL

Package the app with its runtime and dependencies into an **image**. Run that image anywhere as a **container**.

~/backend-day/12-docker $

© Ashan Dimantha

What is Docker, and why use it?

WHAT

A tool that packs your app **and everything it needs** into one box that runs the same anywhere.

↓ runs on any machine with Docker

WHY

Same everywhere: laptop, test server, AWS
No more “works on my machine”

Setup in one command
`docker run mongo`, no installer

No clashes: Node 18 and Node 22 apps side by side
Each box brings its own versions

Easy to ship and scale
Need 3 copies? Start 3 containers

Like a shipping container: one standard box, any ship, train or truck

© Ashan Dimantha

Image or container: what's the difference?

IMAGE = A SEALED MEAL KIT

- Recipe plus every ingredient
- Read-only, versioned, shareable
- Like a class

CONTAINER = THE DISH COOKING

- A running copy of the kit
- Start, stop, throw away
- Like an object; many per image

Containers share the host's kernel. Lighter than a virtual machine, less isolated.

© Ashan Dimantha

How do we write the recipe?

```
# Dockerfile
FROM node:[lts]-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER node
EXPOSE 3000
CMD ["node", "app.js"]
```

BASE

Pin a version. Verify the current LTS tag.

CACHE

Dependencies first, so code changes don’t reinstall them.

.dockerignore

node_modules · .env · .git

© Ashan Dimantha

Build once, run anywhere: what are the commands?

```
$ docker build -t books-api:1.0 .
$ docker run -d -p 3000:3000 --env-file .env books-api:1.0
$ docker ps
$ docker logs -f [container-id]
```

`-p 3000:3000`

your laptop's port : the container's port

`--env-file .env`

config injected at run time, not baked in

© Ashan Dimantha

ACTIVITY 12 · Docker · you drive · 3 min

Can you start the API and the database by hand? You tell me what to click

BY HAND

1 · create a network

2 · start the database

3 · start the API

OR

docker compose up

↺ reset

network	db	api	commands
off	off	off	0

Count the commands. Then count again with Compose.

© Ashan Dimantha

ACTIVITY 12 · Mini-quiz · 3 min

Docker: quick check

1 · Which one actually runs: the image or the container?	The container
2 · Why copy package.json before the rest of the code?	Layer cache: faster rebuilds
3 · Should .env be copied into the image?	No: inject config at run time

Shout-outs welcome

© Ashan Dimantha

ACTIVITY 13 · Cloud · intro

15:15 → 15:35 · 20 min

Your laptop is the server. What could go wrong?

I CLOSE MY LAPTOP

The API goes offline

Cloud: servers run 24/7

10,000 USERS ARRIVE

Everything slows down

Cloud: add servers in minutes

MY DISK DIES

All the books are gone

Cloud: managed, backed-up DBs

USERS IN LONDON

Every request crosses the world

Cloud: regions near users

SO WHAT IS “THE CLOUD”?

Renting someone else’s computers over the internet, and **paying only for what you use**.

AWS · Microsoft Azure · Google Cloud

Think 30 seconds, then call out one problem

© Ashan Dimantha

How much of the server do you want to own?

VIRTUAL MACHINE

Amazon EC2

You patch the OS and run Docker yourself.

Like owning a car

CONTAINERS · OUR PICK

ECS on Fargate

You bring the image; AWS runs it.

Like leasing a car

FUNCTIONS

AWS Lambda

You bring a function; pay per call.

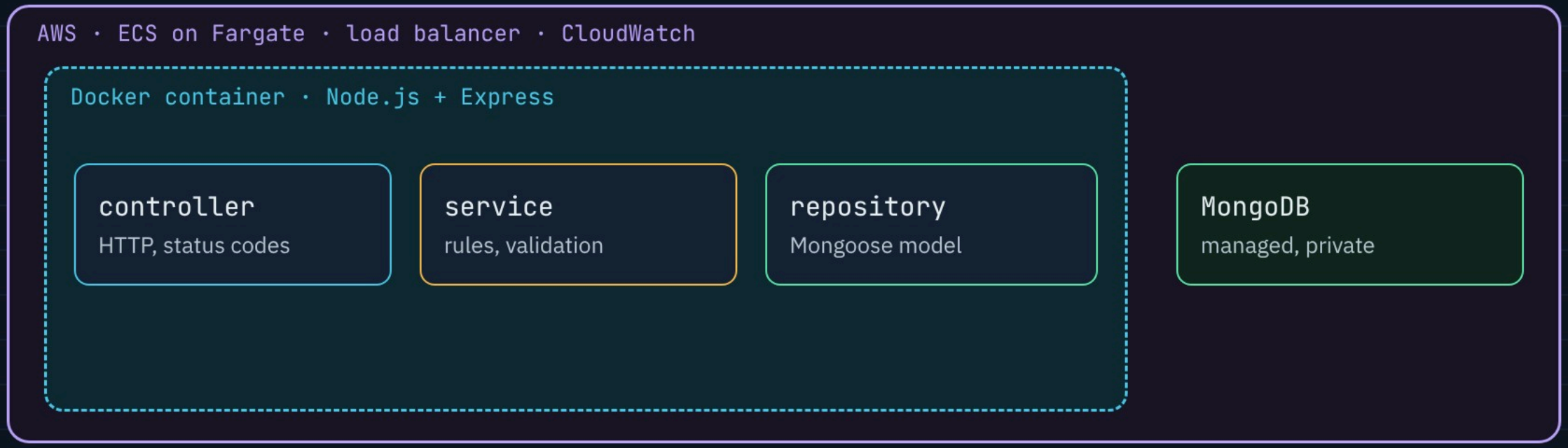
Like a taxi

← more control · · ·

WRAP-UP 15 · The day in one picture

15:50 → 16:00 · 10 min

What did we build today?



Around it all: DNS · TLS · HTTP · auth · rate limits · secrets · logs, metrics, traces · an SLO

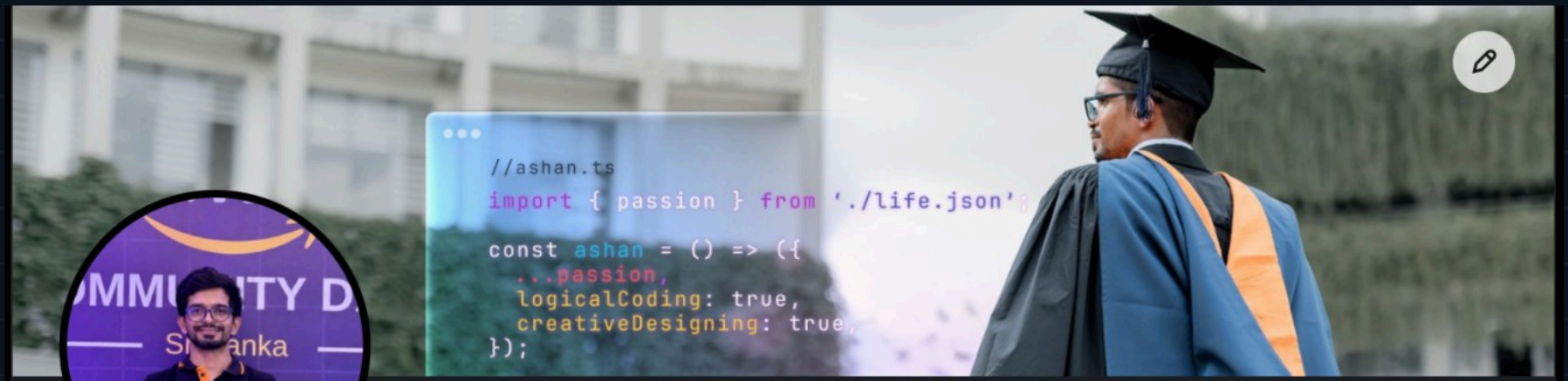
~/backend-day/15-wrap $

© Ashan Dimantha

Annonymous Feedback

© Ashan Dimantha

Connect with me



Ashan Dimantha

Software Engineer @Fortude | Technical Mentor @STEMLink | UI/UX & Cloud Enthusiast

Ragama, Western Province, Sri Lanka · **Contact info**

Fortude + 2 more

University of Plymouth

© Ashan Dimantha

GitHub repository

© Ashan Dimantha

$ curl -X POST /questions

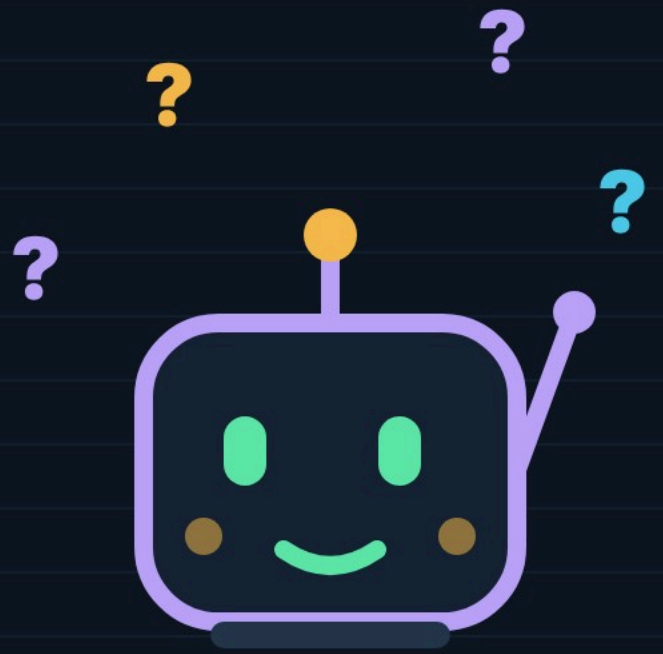
Q&A

Ask anything. No question is too basic.

Something from today that didn’t click

Careers, tools, “how do real teams do this?”

Stump the presenter: I’ll follow up by email



201 Created: every question gets a response

© Ashan Dimantha

$ exit 0 # thanks for a great day

Thank you!

KEEP LEARNING

MDN Web Docs on HTTP · Node.js and Express docs · MongoDB University

OWASP Top 10 and the OWASP Top 10 for LLM Applications

OpenTelemetry docs · AWS Well-Architected Framework

Pip: “200 OK. See you in production.”

© Ashan Dimantha

APPENDIX Facilitator outline · 1 of 2

Morning: what happens where?

Time	Section	Slides	Mode	Interactions
09:00	00 Welcome	1–5	Activity	Comfort poll, weirdest-bug think-pair-share
09:10	01 Intro to back-end	6–14	Theory	Face a question, restaurant analogy, true/false quiz
09:30	02 Runtime & APIs	15–24	Theory	Predict the output, breaking-or-safe quiz
09:55	03 API design	25–35	Theory	Poll, analogy match, pick-the-style TPS, quiz
10:25	Break	36	Break	Looping animation
10:40	04 Networking	37–43	Theory	dig micro-task, which port?, quiz
11:00	05 HTTP	44–51	Theory	Which status code?, quiz
11:20	06 Lab 1	52–66	Lab	7 steps, spot the bug, hidden solution, debrief
12:05	Lunch	67	Break	Lab 2 database prep

Every slide’s tag and header name its mode and interaction. Speaker notes hold the timing.

© Ashan Dimantha

APPENDIX Facilitator outline · 2 of 2

Afternoon: what happens where?

Time	Section	Slides	Mode	Interactions
12:50	07 Layers	68–77	Theory	Animated flow, which layer?, quiz
13:15	08 Databases	78–85	Theory	Poll, predict the query plan, quiz
13:35	09 Lab 2	86–100	Lab	11 steps, hidden solution, debrief
14:05	Break	101	Break	Looping animation
14:20	10 Observability	102–107	Theory	Span waterfall, which signal?
14:35	11 Security & AI	108–116	Theory	Spot the security bugs, quiz
14:55	12 Docker	117–123	Theory	Live demo, predict the failure, quiz
15:15	13 Cloud	124–129	Theory	Poll, whose job?
15:35	14 Q&A	130	Activity	Stump the presenter
15:50	15 Wrap-up	131–133	Activity	Build-up recap, Monday commitments

Appendix: 134–139 · outline, activity index, analogy sheet, worksheet

© Ashan Dimantha

APPENDIX Activity index

Where is each activity card?

4 Icebreaker comfort poll
5 Weirdest bug · think-pair-share
14 Client/server true or false
19 Predict the output · event loop
24 Breaking change or safe?
25 API style poll
31 Analogy match · API styles
34 Pick the style · think-pair-share
35 API styles quiz
42 Which port?
43 Network detective quiz
50 Which status code?
51 HTTP quiz
60 Spot the bug · Express
66 Lab 1 debrief
75 Which layer?
77 Layers quiz
78 Database poll
83 Predict the query plan
85 Embed or reference?
100 Lab 2 debrief
107 Which signal?
108 Spot the security bugs
116 Security & AI quiz
122 Predict the failure · Docker
123 Docker quiz
124 Cloud poll
129 Whose job?
130 Stump the presenter
132 Monday commitments

Each card (goal · instructions · time · expected answers) is in that slide's speaker notes.

© Ashan Dimantha

APPENDIX Analogy reference · 1 of 2

Which analogy goes with which idea?

Client & server → diners and the kitchen · slide 9

Request lifecycle → post office: address, counter, parcel · 11

Event loop → one chef with oven timers · 17

API contract → a restaurant menu · 23

SOAP · REST · GraphQL → government form · menu · build-your-own bowl · 31

IP address & port → building address and flat number · 38

DNS → phone contacts and directory enquiries · 39

TCP vs UDP → registered mail vs postcards · 40

TLS → passport check, then a sealed envelope · 41

Idempotency → lift button vs vending machine · 47

Status code classes → airport announcements · 48

Controller · service · repository → waiter · chef · pantry keeper · 70

Relational vs document → linked spreadsheet tabs vs complete cards · 80

Indexes → library catalogue; phone book order · 82–83

Continued on the next slide

© Ashan Dimantha

APPENDIX Analogy reference · 2 of 2

Which analogy goes with which idea?

Logs · metrics · traces → ship's logbook · car dashboard · parcel tracking · slide 103
SLO & error budget → a bus timetable promise · 106
Input validation → airport security scan with an allowlist · 109
Authentication vs authorization → hotel reception and key card · 110
Secrets → house keys: door, doormat, neighbour, key safe · 111
Rate limiting → a club bouncer · 112
LLM API in a back end → a translation agency's front desk · 113
LLM cost control → agreeing a taxi fare before the ride · 114
Image vs container → sealed meal kit vs the dish cooking · 118
Docker Compose → a kitchen's order sheet · 121
localhost in a container → shouting in your own flat · 122
Shared responsibility → renting a flat · 125
VM · containers · functions → own a car · lease a car · take a taxi · 126
Environment config → same recipe, different kitchens · 128

28 concepts · at least one analogy each

© Ashan Dimantha

WORKSHEET Back-End Engineering for Software Engineers · [Date]

What I'll change on Monday

Name _____

1 · this week _____

2 · this month _____

3 · with my team _____

Ideas: request ids in logs · an OpenAPI contract · rate-limit login · move secrets to a secrets manager · an SLO for one endpoint

Print me · photograph me · put a reminder in your calendar for Monday

© Ashan Dimantha